A Project Report

On

“MP Online Chatbot”

Submitted by

Uday Kushwaha (B2255R10106006)

2026

CERTIFICATE

This certify that the project report entitled "**FRESH HARVEST**" submitted by partial fulfillment of the requirement for the degree of Bachelor of Technology in Computer Science & Engineering in **June-December 2025.** AKS University, Satna is a bona-fide project work carried out by **Uday Kushwaha, B2255R10106006,** under my supervision. The supervisor has approved the subject of the project report. This is also to certify that it is his/her original work and no part of this project is report has been submitted for any other degree/diploma.

All the assistance and help received during the course of the investigation has been duly acknowledged.

1. I am satisfied that the report presented by **Uday Kushwaha**is worthy of consideration for award of the degree.
2. I certify:
   i) That he/she pursued the prescribed course of research.
   ii) That he/she bears good moral character.

Place: AKSU, Satna

Date:…/…./……

…… **……**

External **Guide-……**

……. ……

Dean **Head of Department**

Prof. (Dr.) Akhilesh A. Waoo **Dr. Chandra Shekhar Gautam**

Dean Computer Science and Engineering (Head CSE, AKS University)

SELF DECLARATION

I hereby declare that the work presented in this project entitled **"Fresh Harvest"** towards the partial fulfilment of the requirement for the award of **Degree in B.Tech** in Department of Computer Science, **AKS University, Satna (M.P.)** is an authentic record of my own work.

I have not submitted the matter embodied in the project for the award of any other degree or diploma to any other institute or university.

……..

Signature of Candidate

Uday Kushwaha

ACKNOWLEDGEMENT

It is a great opportunity for me in taking this opportunity to express my sincere thanks and appreciation to my supervisor and Head of Department **Dr. Chandra Shekhar Gautam**. I consider myself lucky enough to have such a great project.

At this moment of accomplishment, first of all, I pay homage to my guide, Dr. Akhilesh A. Waoo, HoD, CSE - Dr. Chandrashekhar Gautam from AKS University Satna (M.P.). This work would not have been possible without his guidance, support, and encouragement. Under his guidance, I successfully overcame many difficulties and learned a lot.

I would also like to express my sincere thanks to OS3 Infotech Private Limited, where I worked on the project “Linux Server Hardening with Ansible”. The company not only provided me the required technical environment but also gave me valuable exposure to real-world infrastructure, automation practices, and industry-level standards. I am grateful to the entire team for their continuous support, guidance, and for allowing me to learn and grow in a professional setup. The organization also ensured that all client-related information remains completely confidential and is not exposed at any stage.

I am deeply and forever indebted to my parents for their love, support, and encouragement Throughout my entire life.

Permission Letter

Completion Letter

ABSTRACT

Project Title: Intelligent FAQ Chatbot for MPOnline using Retrieval-Augmented Generation (RAG)

In the rapidly evolving landscape of digital governance, the effective dissemination of information regarding public services is a critical challenge. The FAQ Chatbot for MPOnline is an advanced conversational AI solution designed to automate citizen support and provide accurate, 24/7 assistance for a wide range of government services, such as certificate applications, licensing, and grievance registration.

Traditional chatbots often rely on static, keyword-based responses or suffer from "hallucinations" when using large language models (LLMs) in an unconstrained manner. To address these limitations, this project implements a Retrieval-Augmented Generation (RAG) architecture. This system utilizes a FastAPI backend and a React-based frontend, integrated with Google Gemini AI for natural language understanding and embedding generation.

A core feature of the system is its ability to ingest and process official documents, including PDFs and images, using Tesseract OCR. These documents are segmented into contextual chunks and stored in a MongoDB Vector Store, enabling semantic search capabilities. When a user submits a query, the system retrieves only the most relevant document sections and provides them as context to the Gemini LLM, ensuring that the generated responses are strictly grounded in official sources and factually accurate.

The results demonstrate a significant improvement in response accuracy and user engagement. By providing a secure, scalable, and multilingual interface, the chatbot reduces the operational burden on manual helpdesks while empowering citizens with instant access to verified information. This project showcases the potential of combining RAG pipelines with OCR and vector databases to create reliable, high-performance AI tools for the public sector.

Keywords: Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), Gemini AI, FastAPI, MongoDB Atlas Vector Search, OCR, Digital Governance, Citizen support.

TABLE OF CONTENTS

- Introduction……..……**9-10**
- Objective of The Project…….……..**11-12**
- Scope of The Projects…..…… ……**13-19**
- Definition of Problem…… ……..**20-22**
- Benefit of The Projects …….**23-25**
- Main Module…….……**26-28**
- Technical Overview …….…….**29-32**
- System Configuration…..……**33-35**
- Block diagram …… …….…….……**36-38**
- Project Explanation ……..…….....**40-43**
- Sample Code with Explanation …….……**44-47**
- Result & Analysis …..……**48-51**
- Software Testing …..……**52-56**
- Conclusion …….....…….….…….....**57-58**
- Bibliography / References ……..…….**59**
- Appendix – Student Profile……**60-61**
- Appendix – Plagiarism Report……**62**
- Appendix – PPT Handouts ……..……..…..**63-70**

Introduction

The Chatbot project is a full-stack web application designed to provide users with a simple, efficient, and intelligent platform for interacting with an automated conversational system. The primary aim of this project is to bridge the gap between users and digital services by offering a smart interface where queries can be asked, processed, and answered instantly. With the increasing demand for automation and the growing reliance on digital communication, the chatbot focuses on delivering both convenience and accuracy to its users.

This project follows a complete full-stack development approach, integrating a responsive front-end, a secure back-end, and a reliable database system. The front-end is developed using HTML, CSS, and JavaScript, ensuring a clean user interface and smooth interaction experience. On the back-end, Node.js and Express.js are used to manage server operations, process user inputs, and maintain application logic. The project also utilises MongoDB as the database, allowing efficient storage and retrieval of user queries, responses, and interaction history.

The chatbot system has been built with the goal of creating a user-friendly and scalable solution. Features such as user input handling, real-time response generation, intent recognition, and conversation management make the platform practical and close to real-world applications. Overall, this project demonstrates skills in designing, developing, and deploying an intelligent full-stack application, while addressing an important need of today's digital communication systems.

In today's digital era, technology plays a vital role in transforming traditional communication methods into more efficient, automated, and accessible systems. The chatbot project is developed with this vision of modernisation, aiming to revolutionise the way users interact with digital platforms. With the rapid growth of online services and increasing expectations for instant responses, there is a strong need for an organised system where user queries can be handled quickly and accurately. The chatbot attempts to meet this need by offering a full-stack application that provides automated, reliable, and real-time assistance.

The chatbot is designed as a complete conversational platform, focusing on intelligent interaction and user satisfaction. Unlike traditional systems, this project emphasises accuracy, speed, and continuous interaction. The platform enables users to ask questions, receive relevant responses, and interact naturally with the system. By doing so, the system not only

improves user experience but also reduces manual workload and enhances efficiency in digital communication.

This project follows a structured full-stack development approach, combining a visually appealing front-end with a powerful back-end and a secure database. The front-end interface is created using HTML, CSS, and JavaScript, ensuring that the chatbot interface is both responsive and easy to use. Modern design principles have been applied to maintain clarity, simplicity, and smooth conversation flow. Clean layouts, organised chat windows, and intuitive input fields help users interact effortlessly with the system.

On the server side, Node.js and Express.js are used to handle request processing, API integration, and chatbot logic. The back-end ensures that all operations—such as query processing, response generation, and user interaction handling—are executed efficiently and securely. This structure allows the application to process user inputs quickly and deliver accurate responses in real time. MongoDB is used as the database system to store conversation logs, user data, and response patterns, making the system robust and scalable for future enhancements.

The purpose behind developing this chatbot project goes beyond creating a basic conversational system. It aims to demonstrate the practical application of full-stack development, integration of intelligent systems, and effective data management. Through this project, various technical concepts such as RESTful APIs, routing, natural language processing basics, session handling, and CRUD operations have been implemented. The system also includes features such as automated responses, query handling, conversation flow management, and structured data storage.

The chatbot project also highlights the importance of intelligent automation in modern systems. By providing instant responses to user queries, it reduces human effort, improves efficiency, and enhances user satisfaction. In addition, users benefit from quick access to information without delays. The platform has been designed to support future enhancements such as AI-based learning, voice interaction, multilingual support, sentiment analysis, and integration with external services .

Objective of the Project

The Chatbot project has been developed with the primary aim of creating a modern, efficient, and user-friendly digital platform for automated communication and intelligent query handling. The objective of this platform is not only to provide instant responses to user queries but also to enhance accessibility, efficiency, and user experience in digital interactions. The following objectives outline the vision, purpose, and technical goals that guided the development of the project. These objectives collectively form the foundation on which the entire system has been designed and implemented.

1. To Create a User-Friendly Conversational Interface

One of the main objectives of the chatbot project is to develop an interactive platform where users can easily communicate with the system through natural language. The chatbot is designed with simplicity in mind so that users of all age groups and technical backgrounds can interact without confusion. The objective focuses on providing a clean chat interface, readable text, smooth message flow, and an intuitive design. By doing so, the system ensures that users have a comfortable and seamless conversational experience every time they interact with the chatbot.

2. To Provide Instant and Accurate Responses to User Queries

The chatbot aims to deliver quick and relevant responses by processing user inputs efficiently. This objective focuses on reducing waiting time and improving user satisfaction by ensuring real-time interaction. The system is designed to understand user queries, analyse input patterns, and generate meaningful responses. By doing so, the chatbot enhances communication efficiency and provides reliable assistance to users across different use cases.

3. To Implement a Complete Full-Stack Chatbot Solution

A major objective behind this project is to demonstrate the integration of front-end, back-end, and database technologies in a fully functional chatbot application. The project has been developed using HTML, CSS, and JavaScript for the front-end, Node.js and Express.js for the back-end server, and MongoDB for data storage. The objective here is not only to build a working chatbot but also to showcase technical knowledge of the full-stack development process—from user interface design to server logic and persistent data management.

4. To Ensure Secure and Efficient Data Handling

A crucial objective of the chatbot project is to maintain the privacy and security of user

interactions and stored data. The system includes features such as secure API communication, input validation, and protected storage of conversation logs. Operations such as query processing, response generation, and user data handling are managed securely to prevent unauthorised access or misuse. Data validation, error handling, and secure communication between the client and server are essential components of this objective.

5. To Develop a Structured and Intelligent Response Management System

The chatbot aims to maintain a well-organised response system that allows it to handle various types of queries efficiently. The project includes predefined responses, dynamic response generation, and structured data storage to ensure that users receive accurate and relevant answers. This objective focuses on creating a system that is simple for users yet capable of handling complex queries. The response structure is designed to be easily expandable, allowing future integration of AI-based learning and advanced conversational capabilities.

6. To Provide a Smooth and Continuous Conversational Workflow

Another important objective of the chatbot project is to ensure a smooth and uninterrupted conversation flow. Features such as real-time messaging, context handling, and continuous interaction have been included to improve user experience. The goal is to ensure that the entire conversation process is natural and efficient—from user input to system response. This objective also highlights the importance of maintaining accuracy, clarity, and consistency in chatbot responses at every step of interaction.

Scope of the Project

The Chatbot project has been developed with the aim of creating a modern and efficient digital platform for automated communication and intelligent query handling. The scope of the project extends far beyond building a simple conversational interface; it incorporates technical, functional, social, economic, and future-oriented aspects that collectively define the potential and usefulness of the system. This section provides a detailed explanation of the broad and extensive scope of the chatbot project, describing what the system currently offers and what it can achieve in the future.

1. Technical Scope

The technical scope defines the technologies used, the system architecture, and the overall technical strength of the chatbot system. The project has been designed with modern full-stack technologies that support scalability, flexibility, and high performance.

1.1 MERN Stack Technology Integration

The system is built using:
• HTML, CSS, JavaScript for front-end
• Node.js, Express.js for back-end
• MongoDB for database storage

This integration ensures that the chatbot performs efficiently and is capable of handling real-world requirements such as user query processing, session management, response generation, and API integration.

1.2 Scalability and Future Expansion

The architecture supports future modules such as:
• AI-based natural language processing (NLP)
• Voice-based interaction systems
• Multilingual chatbot support
• Integration with third-party APIs
• Machine learning-based response improvement

The chosen stack allows the system to expand without requiring major restructuring.

1.3 Cross-Platform Compatibility

The chatbot interface is responsive and works on:

• Desktop

• Laptop

• Tablets

• Mobile phones

This ensures accessibility and usability for a wide range of users across different devices.

2. Functional Scope

Functional scope refers to what the chatbot system can perform, the features it offers, and the operations it supports.

2.1 User Interaction and Query Handling

Users can interact with the chatbot by entering queries in natural language. The system processes inputs and provides appropriate responses in real time, ensuring efficient communication.

2.2 Conversation Flow and Response Management

The chatbot is capable of handling structured conversations, maintaining context, and generating meaningful responses based on predefined logic or dynamic input analysis.

2.3 Real-Time Messaging System

Users can:

• Send queries instantly

• Receive immediate responses

• Continue conversation seamlessly

This creates a smooth and continuous conversational experience.

2.4 Integration with External Systems

The chatbot can be integrated with websites, applications, or services to provide automated support. In future versions, it can:

• Connect with customer support systems

• Access external APIs

• Provide automated assistance across platforms

3. Social Scope

The chatbot project has a meaningful social impact as it improves accessibility to information and enhances communication efficiency.

3.1 Improving Digital Communication

The chatbot reduces dependency on manual support by providing instant responses, making communication faster and more efficient.

3.2 Enhancing User Experience

Users benefit from:

• Quick responses

• 24/7 availability

• Easy interaction

This improves overall satisfaction and accessibility.

3.3 Supporting Digital Transformation

Chatbot systems help organisations and communities adopt digital tools, improving service delivery and operational efficiency.

4. Economic Scope

The chatbot project has the potential to become a valuable commercial solution.

4.1 Market Potential

The demand for chatbot systems is rapidly increasing in industries such as customer support, education, and e-commerce. The project can be expanded into a business model by adding:

• Customer support automation

• AI-powered assistance

• Subscription-based chatbot services

4.2 Low Operational Cost

Chatbots reduce the need for human support staff, lowering operational costs. The system can handle multiple users simultaneously without additional expenses.

4.3 Revenue Generation

The project has multiple revenue opportunities:

• Subscription-based services

• API integration services

• Business automation solutions

• Premium chatbot features

5. Educational Scope

From an academic perspective, the chatbot project demonstrates full-stack development, system design, and intelligent application development.

5.1 Real-World Application Development

The project shows how different components interact:

• Front-end

• Back-end

• Database

• APIs

5.2 Implementation of Modern Coding Practices

Including:

• MVC architecture

• RESTful APIs

• Secure authentication

• Modular coding

5.3 Documentation Skills

The project also helps in practising:

• Report writing

• System analysis

• Software design

• Deployment methods

6. Operational Scope

The operational scope explains how the system functions internally.

6.1 Data Management

All user queries, responses, and interaction history are stored securely and can be retrieved efficiently.

6.2 System Performance

The technology stack ensures:

• Fast response time

• Low latency

• Smooth interaction

• Efficient API communication

6.3 Security Management

The system includes:

• Secure API handling

• Input validation

• Protection against unauthorised access

• Safe data storage

7. Future Scope

The chatbot project has strong future potential.

7.1 AI-Based Chatbot Enhancement

The system can be upgraded with:

• Natural Language Processing (NLP)

• Machine learning algorithms

• Context-aware responses

7.2 Voice-Based Interaction

Users can interact using voice commands, improving accessibility.

7.3 Mobile Application Development

A dedicated mobile app can increase reach and usability.

7.4 Multilingual Support

The chatbot can support multiple languages for wider accessibility.

7.5 Sentiment Analysis

The system can analyse user emotions to provide better responses.

7.6 Advanced Dashboard

Admins can manage:

• Conversations

• User data

• Performance analytics

8. Commercial Deployment Scope

The chatbot system .

Problem Definition

During the development of the chatbot full-stack project, several challenges and problems were encountered at different stages of planning, designing, and implementing the system. These problems were both technical and non-technical in nature. Understanding these problems is important because they helped shape the final structure of the project and provided opportunities to apply problem-solving skills, improve system efficiency, and enhance overall project quality. This section discusses the major problems faced during the project and how they influenced the development journey.

1. Understanding User Requirements

One of the first challenges was understanding what users truly expect from a chatbot system. Users want:

• Instant and accurate responses
• Natural and easy conversation flow
• Quick interaction without delays
• Reliable and meaningful answers
• Simple and intuitive interface

The difficulty was converting these expectations into clear technical features. At the beginning, it was not easy to decide:

• How intelligent the chatbot should be
• Which features should be implemented first
• How to maintain simplicity while ensuring effectiveness

This required multiple revisions, brainstorming sessions, and re-planning before finalising the system design.

2. Designing a Clean and Responsive User Interface

Another major problem was creating a chatbot interface that works smoothly on desktop, laptop, mobile, and tablet screens. A responsive chatbot UI requires:

• Flexible chat layout
• Proper alignment of messages

• Readable fonts and clear chat bubbles
• Simple and uncluttered design

Many issues occurred such as:
• Chat messages overlapping on smaller screens
• Input box not aligning properly
• Text visibility problems
• Chat window not adjusting on mobile view

Fixing these problems required continuous testing and refining CSS and layout design until the interface became stable and user-friendly.

3. Handling Data Properly in PostgressSQL

When storing user queries, chatbot responses, and conversation history, several challenges appeared:
• Structuring collections correctly
• Managing conversation logs efficiently
• Avoiding duplicate or redundant data
• Maintaining relationships between user sessions and responses

At first, maintaining clean and organised data was difficult. Some issues included:
• Incomplete conversation records
• Inconsistent field naming
• Difficulty tracking user interactions

Proper schema design, validation, and structured data storage were later implemented to resolve these issues.

4. Implementing Query Processing and Security

Handling user queries securely and accurately was one of the biggest challenges in the project. This required:
• Processing user input correctly
• Preventing invalid or harmful inputs

• Managing sessions securely
• Ensuring accurate response generation

Mistakes in this process could lead to incorrect responses or system misuse. During development, issues such as:
• Incorrect input processing
• Session handling errors
• Responses not matching user queries
• Improper validation rules

were encountered. These were resolved by improving input validation, refining logic, and enhancing error handling mechanisms.

5. Managing Back-End Logic and APIs

Building a stable back-end using Node.js and Express.js required careful handling of chatbot logic and API routes. Some problems included:
• API routes not responding correctly
• Incorrect response generation
• Difficulty connecting user input with backend logic
• Errors not being displayed properly

At times, the server faced issues due to incorrect coding, missing parameters, or faulty logic. Debugging tools and console logging helped identify and fix these problems gradually.

6. Ensuring Smooth Communication Between Front-End and Back-End

Connecting the chatbot interface with the server was another challenge, especially when sending and receiving data using APIs. Issues included:
• Incorrect API endpoints
• CORS-related errors
• Undefined variables
• Delay in response generation
• Improper handling of JSON data

These problems caused unexpected behaviour such as:

• Chatbot not responding

• Messages not displaying properly

• Delayed or incorrect replies

Proper API handling, response formatting, and debugging techniques helped resolve these issues and ensured smooth communication between the front-end and back-end systems.

Benefits of the Project

The Chatbot project provides a wide range of benefits, both from a technical perspective and from a user or organisational point of view. As a full-stack platform for automated communication and intelligent query handling, it aims to make interaction with digital systems faster, more efficient, and more accessible. The following points explain the key benefits offered by the project.

1. Convenience and Accessibility

One of the major benefits of the chatbot system is the high level of convenience it provides to users. Users can ask questions, receive responses, and interact with the system instantly without the need for manual support. This eliminates delays and improves efficiency, especially during peak usage times or when human assistance is unavailable.

The chatbot is accessible on mobile, tablet, and desktop devices, making it usable anytime and anywhere. This accessibility ensures that users can interact with the system regardless of their location or device, providing a seamless communication experience.

2. Instant Response and Automation

The chatbot helps reduce the dependency on human intervention by providing automated responses to user queries. By handling multiple users simultaneously, the system ensures quick and consistent replies without delays.

This automation improves efficiency, reduces workload, and enhances productivity. Users receive accurate information instantly, making the system highly reliable for real-time interaction.

3. Improved Accuracy and Information Transparency

The chatbot allows users to receive clear and structured responses based on their queries. Information is processed and presented in a logical and understandable manner, ensuring clarity in communication.

The system maintains transparency by providing consistent and relevant answers, reducing confusion and miscommunication. Users can rely on the chatbot for accurate and up-to-date information.

4. Enhances User Experience

The chatbot encourages smooth and natural interaction between users and the system. By providing instant responses and maintaining a continuous conversation flow, it improves the overall user experience.

This benefit extends to users of all age groups and technical backgrounds, making the system easy to use and highly interactive. The chatbot ensures a comfortable and engaging communication environment.

5. Time-Saving and Efficient Communication

The chatbot significantly reduces the time required to search for information or wait for responses.

With features such as:

• Instant messaging

• Automated responses

• Continuous conversation flow

• Quick query resolution

Users can complete their interactions quickly and efficiently. This results in a smooth and productive communication experience compared to traditional methods.

6. Educational Value for Developers

From an academic perspective, the chatbot project is highly beneficial for learners and developers. The project demonstrates:

• Full-stack development implementation

• Database management

• API communication

• Real-time data processing
• Front-end and back-end integration

Students gain practical experience by working with modern technologies such as Node.js, Express.js, and MongoDB. This enhances their technical knowledge and prepares them for real-world development challenges.

7. Scalable and Future-Ready System

The chatbot system has a strong technical foundation, allowing new features to be added easily in the future. The benefits of scalability include:
• Ability to handle more users simultaneously
• Integration with AI and machine learning models
• Support for voice-based interaction
• Expansion into multilingual systems
• Integration with external platforms and services

Because the architecture is well-structured, the system can evolve into a fully intelligent conversational platform suitable for various industries and applications.

Main Modules

The Chatbot project is organised into several core modules that define the overall functionality and structure of the system. Each module plays an essential role in delivering a smooth, efficient, and user-friendly conversational platform for automated query handling. These modules work together to handle different operations, including user interaction, response processing, conversation management, and data handling. The following sections describe the main modules in detail.

1. User Interaction Module

The User Interaction Module is responsible for handling all communication between the user and the chatbot system. It manages how users input their queries and how the chatbot responds in a structured conversational format. This module ensures that users can interact with the system easily and efficiently.

Key Functions:

• **User Input Handling**

Users can enter queries through a chat interface using text input. The system captures and processes the input in real time.

• **Message Display**

The chatbot displays both user messages and system responses clearly in a conversational format.

• **Session Handling**

Each user session is managed to maintain continuity in conversations.

• **Interface Management**

The chat interface is designed to be simple, responsive, and easy to use across different devices.

This module forms the foundation of user interaction and ensures a smooth communication experience.

2. Response Processing Module

The Response Processing Module handles how the chatbot understands and generates responses to user queries. It is responsible for analysing input and providing meaningful outputs.

Key Functions:

• **Query Analysis**

The system processes user input to understand the intent behind the query.

• **Response Generation**

Based on predefined logic or dynamic processing, the chatbot generates appropriate responses.

• **Keyword Matching / Logic Handling**

The chatbot uses conditions or patterns to match queries with relevant responses.

• **Backend Processing**

All query handling logic is managed on the server side using Node.js and Express.js.

This module ensures that users receive accurate and relevant responses during interaction.

3. Conversation Management Module

The Conversation Management Module maintains the flow and continuity of communication between the user and the chatbot. It ensures that conversations are structured and meaningful.

Key Functions:

• **Context Handling**

The chatbot maintains context to provide relevant responses based on previous interactions.

• **Conversation Flow Control**

Ensures smooth transition between messages and maintains logical flow.

• **Session Tracking**

Tracks ongoing conversations for better response accuracy.

• **Error Handling**

Provides fallback responses when the chatbot cannot understand the query.

This module enhances the overall conversational experience and makes interactions more natural.

4. Data Management Module

The Data Management Module is responsible for storing, retrieving, and managing all chatbot-related data. It ensures efficient handling of user queries and system responses.

Key Functions:

• **Data Storage**

User queries, responses, and conversation logs are stored securely in MongoDB.

• **Data Retrieval**

Stored data is fetched quickly when needed for processing or analysis.

• **Database Structure**

Efficient schema design ensures organised and scalable data storage.

• **Data Security**

Sensitive data is handled securely to prevent unauthorised access.

This module ensures that the chatbot system remains reliable and efficient in handling data.

5. Admin / Management Module (Future-Ready Module)

While optional in the current version, the Admin Module is an important part of the chatbot system's future scope. It allows administrators to monitor, manage, and improve chatbot performance.

Possible Functions:

• Monitor user interactions and conversation logs
• Update chatbot responses and logic
• Analyse system performance and usage data
• Manage user sessions and system configurations

This module supports system improvement, scalability, and long-term maintenance of the chatbot platform.

Technical Overview

The Chatbot project is developed as a complete full-stack application designed to automate communication and provide intelligent responses to user queries. The technical architecture of the system follows a modern, scalable, and modular approach, integrating front-end, back-end, and database technologies to deliver an efficient and user-friendly conversational platform.

1. Front-End Technologies

The front-end of the system is built using REACT, along with optional libraries such as Bootstrap or Tailwind CSS for responsive design. The user interface is designed to offer a clean, intuitive, and seamless conversational experience. Key front-end features include:

• **Responsive Chat Interface:** Ensures the chatbot works smoothly on mobile devices, tablets, and desktops.
• **Dynamic Chat Window:** Displays user messages and chatbot responses in real time.
• **Input Field and Message Handling:** Allows users to send queries easily and interact continuously.
• **User Authentication UI (Optional):** Login and signup forms connected with backend validations.
• **Real-Time Interaction:** Messages update instantly without page reload using JavaScript.
• **Modern UI Elements and Animations:** Enhance user engagement and improve conversational experience.

The front-end communicates with the server using RESTful APIs, enabling smooth interaction and real-time messaging.

2. Back-End Technologies

The back-end is developed using Node.js with Express.js, forming the core logic of the chatbot system. Express.js provides routing, middleware support, and request handling required for chatbot functionalities. Key components include:

• **Query Processing Module:** Handles user inputs and processes them to generate appropriate responses.
• **Response Management System:** Matches user queries with predefined or dynamic

responses.

• **API Development:** REST APIs ensure smooth communication between the front-end and database.

• **Session Handling / JWT Tokens:** Maintains user sessions and conversation continuity.

• **Integration Capability:** Allows connection with external APIs or AI-based services.

The backend architecture follows modular coding principles, improving maintainability and scalability.

3. Database Layer

The database is handled using MongoDB, managed through MongoDB Compass for GUI interaction. MongoDB's flexible document structure makes it ideal for storing chatbot-related data such as user queries, responses, and conversation logs. Technical highlights include:

• Collections for Users, Conversations, and Responses
• Efficient Query Handling and Fast Data Retrieval
• Scalable Storage for Large Conversation Data
• Schema Modelling Using Mongoose (if implemented)

The database ensures that all interactions are stored securely and can be accessed efficiently when required.

4. System Architecture

The application follows a three-tier architecture:

1. Presentation Layer (Front-end)

o Displays chatbot interface

o Handles user interaction and input

2. Application Layer (Back-end)

o Processes user queries

o Generates responses

o Interacts with the database

3. Data Layer (Database)

o Stores conversation data

o Retrieves responses and logs

o Maintains data consistency

This modular architecture ensures that the chatbot system is robust, scalable, and easy to maintain.

5. Security Features

To ensure safe interaction and protect user data, several security measures are integrated:

• Input Validation to prevent malicious queries
• Secure API communication
• Token-Based Authentication for session management
• CORS Handling for API security
• Protection against unauthorised access

These features make the chatbot system reliable and suitable for real-world deployment.

6. Development Workflow and Tools

The project was developed using a modern toolchain:

• VS Code for coding and development
• Node Package Manager (NPM) for dependency management
• Git/GitHub for version control
• Postman for API testing
• MongoDB Compass for database monitoring

This workflow ensures efficient development, debugging, and system optimisation.

7. Deployment Possibilities

The chatbot system can be hosted on:

• Render / Vercel / Netlify (front-end)
• Railway / Render / Heroku (backend)
• MongoDB Atlas (cloud database)

This scalability ensures that the chatbot can handle a large number of users and can be deployed in real-world environments.

The Chatbot project is developed as a comprehensive full-stack application designed to automate communication, handle user queries intelligently, and provide real-time responses through a conversational interface. The system aims to replace traditional manual interaction methods with an efficient, scalable, and user-friendly digital solution. The technical architecture of the chatbot follows a modern, modular, and layered approach, ensuring high performance, flexibility, and ease of maintenance. By integrating front-end technologies, back-end processing, and a NoSQL database system, the chatbot is capable of handling multiple user interactions simultaneously while maintaining accuracy and responsiveness.

The overall system is designed to support real-time communication, structured data management, and future integration with advanced technologies such as Artificial Intelligence (AI), Natural Language Processing (NLP), and Machine Learning (ML). The architecture ensures that each component works independently yet collaboratively, making the system robust and adaptable for future enhancements.

Project Directory Structure

```
FAQ-Chatbot-MPOnline-main/
│
├── app/ # Backend Application (FastAPI)
│  ├── data/ # Data storage or resources
│  ├── models/ # Database models
│  ├── routes/ # API endpoint routers
│  ├── services/ # Core business logic and services
│  ├── uploads/ # Directory for uploaded files
│  ├── utils/ # Helper utilities and functions
│  ├── config.py # Application configuration settings
│  ├── database.py # Database connection setup
│  ├── dependencies.py # FastAPI dependencies (e.g., auth, DB sessions)
│  ├── main.py # FastAPI application entry point
│  ├── rag_pipeline.py # RAG (Retrieval-Augmented Generation) logic
│  └── __init__.py
│
├── frontend/ # Frontend Application (React/Vite)
│  ├── node_modules/ # Node dependencies
│  ├── src/ # Source code for React (Components, Pages, etc.)
│  ├── index.html # Main HTML template
│  ├── package.json # Node.js dependencies and scripts
│  ├── package-lock.json # Exact versions of Node dependencies
│  └── vite.config.js # Vite build configuration
│
├── alembic/ # Database migration scripts (Alembic)
├── scratch/ # Temporary scratchpads for development
├── venv/ # Python Virtual Environment
│
├── .env # Environment variables (Credentials, keys)
├── .gitignore # Files to ignore in Git version control
├── .uvicornignore # Files to ignore for Uvicorn auto-reload
├── alembic.ini # Alembic configuration file
├── API_DOCUMENTATION.md # Documentation for API endpoints
```

```
├── dev.py # Development script (likely for starting the dev server)
├── README.md # Project description and setup instructions
├── requirements.txt # Python dependencies
└── test_upload.py #
```

System Configuration

1. Hardware Requirements

・Minimum Hardware Requirements

• **Processor:** Dual-Core CPU (Intel i3 or AMD equivalent)
• **RAM:** 4 GB
• **Storage:** 2–4 GB available for project files, dependencies, and database documents
• **Hard Drive Type:** HDD or SATA SSD
• **Graphics:** Integrated graphics
• **Display:** 720p or 1366×768 resolution

Recommended Hardware Requirements

• **Processor:** Intel Core i5/i7 or AMD Ryzen 5/7
• **RAM:** 8–16 GB
• **Storage:** 128 GB or higher SSD
• **Display:** Full HD (1920×1080)
• **Internet:** Stable broadband for fetching NPM packages and connecting with cloud services

2. Software Requirements

· Control Node

- Windows 10 / 11
- Linux distributions (Ubuntu, Fedora, Debian, etc.)
- macOS (10.15 and above)

3.Development Software and Tools

a) Node.js

- Version: **v18 – v20 recommended**
- Provides server-side JavaScript environment
- Includes **NPM** for managing packages and dependencies
- Required for running backend APIs, authentication, cart system, and order logic

b) Express.js

- Lightweight Node.js framework for building backend APIs
- Used for routing, middleware handling, and server configuration

c) Database

- Postgress
- Allows creating collections, viewing documents, and managing user and product data
- Supports SQL document structure suitable for e-commerce items

d) Front-End Tools

- **Framework – React**

e) Version Control

- **Git**
- **GitHub** repository for storing the full-stack project code

f) IDE / Code Editor

- **Visual Studio Code** (recommended)
- Extensions for JavaScript, Node.js, MongoDB, REST API testing, and debugging tools

g) API Testing Tools

- **Postman**
- Used for testing routes like login, signup, add product, add to cart, checkout, etc.

h) Browser Requirements

The system is compatible **with** modern browsers:

- Google Chrome
- Microsoft Edge
- Firefox
- Safari

These browsers support ES6 JavaScript, CSS Flexbox/Grid, and client-side rendering.

4. Server and Deployment Configuration

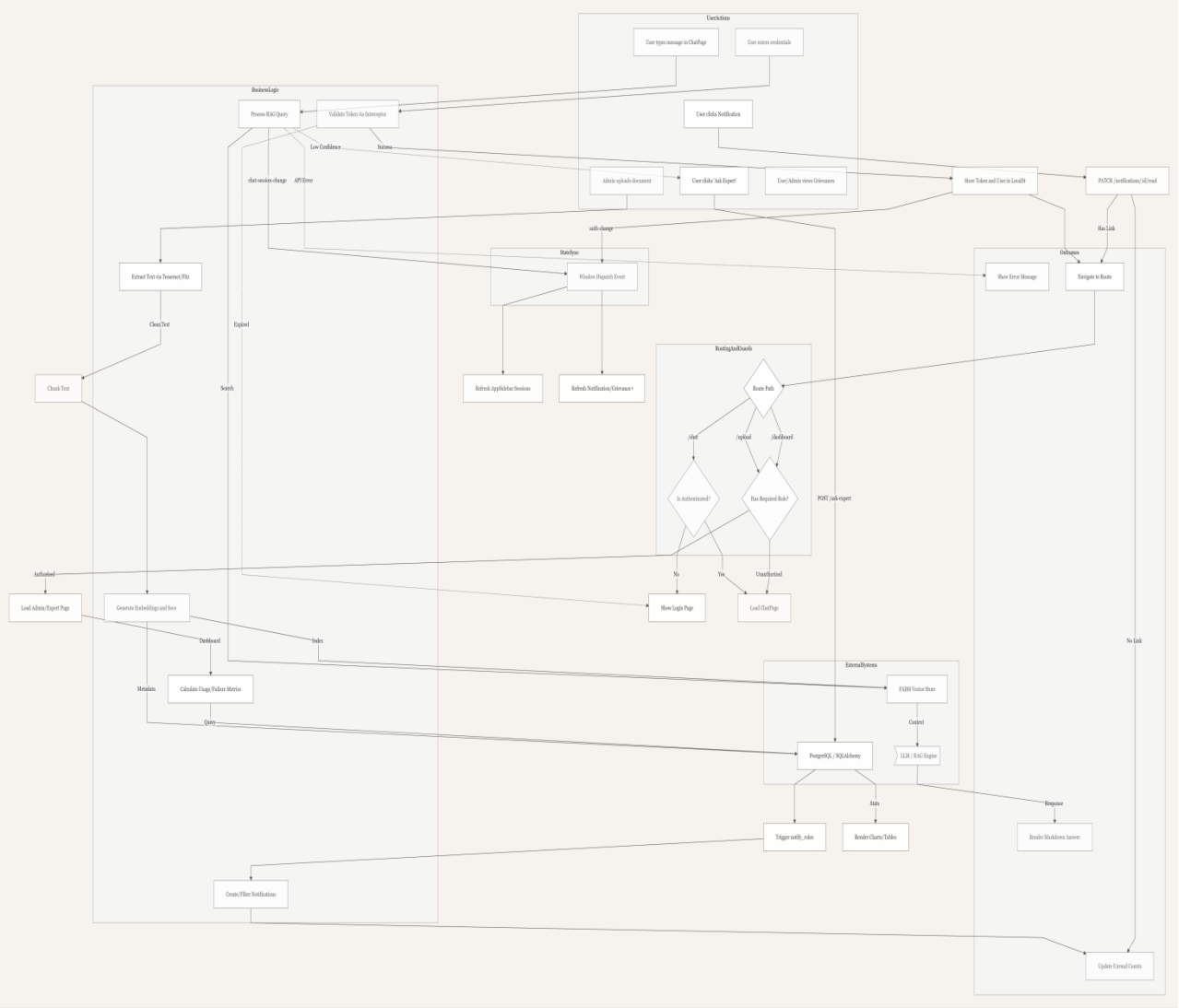
If the project is deployed online, the following configuration is recommended:

- **Backend Hosting:** Render, Railway, or AWS EC2
- **Frontend Hosting:** Netlify, Vercel, or GitHub Pages
- **Cloud Database:** MongoDB Atlas (free tier available)

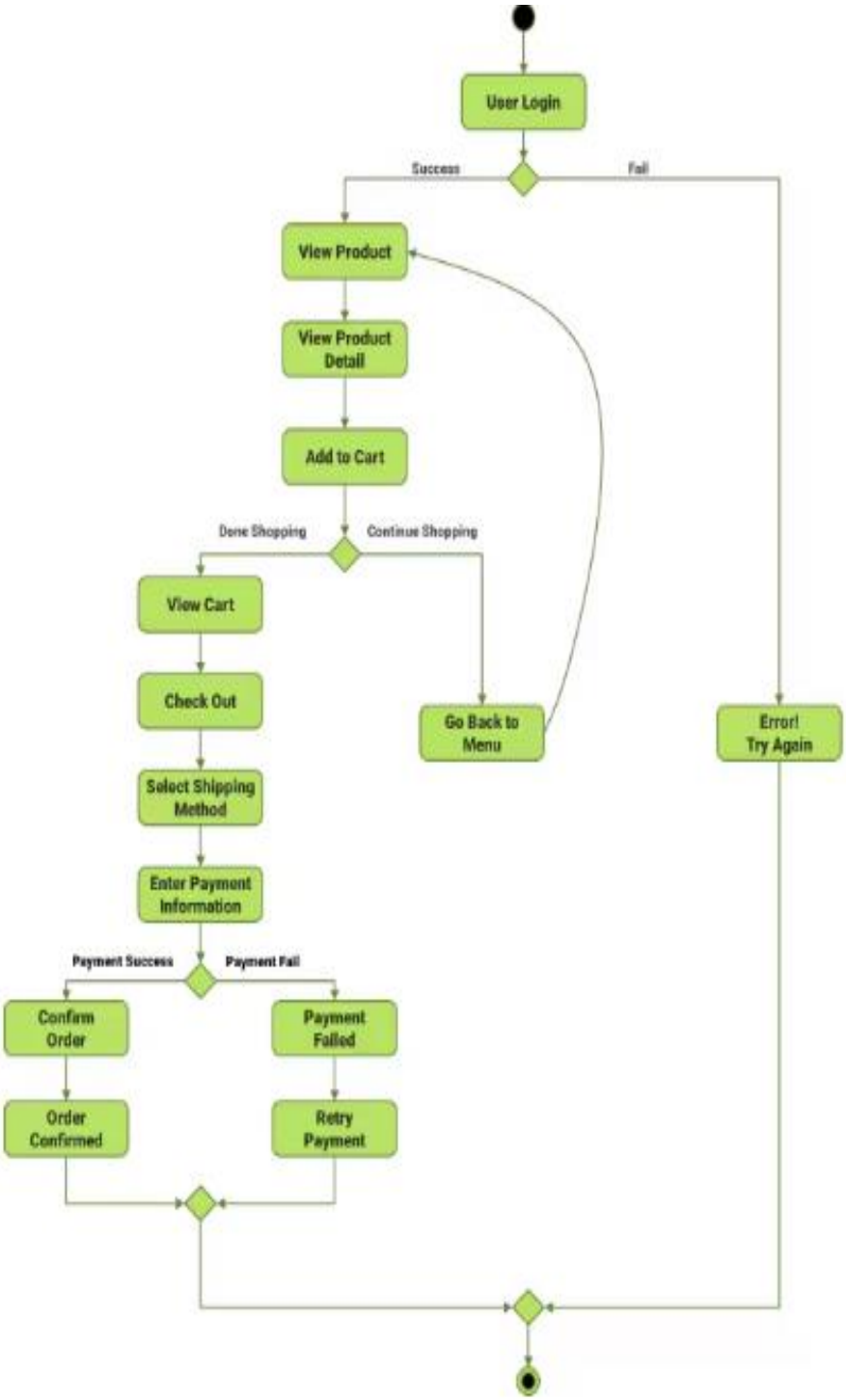
Deployment Requirements

- Node.js installed on server
- HTTPS certificate (optional for security)
- Environment variables for database URI, JWT secret, etc.
- Build scripts for starting backend server
- CORS configuration if frontend and backend are hosted separately

Block Diagram



DFD


User Login
Success
Fail
View Product
View Product Detail
Add to Cart
Done Shopping
Continue Shopping
View Cart
Check Out
Go Back to Menu
Error! Try Again
Select Shipping Method
Enter Payment Information
Payment Success
Payment Fail
Confirm Order
Payment Failed
Order Confirmed
Retry Payment

Project Explanation

The MPOnline FAQ Chatbot is a full-stack web application designed to provide automated, accurate, and efficient responses to user queries based on official MPOnline documents. The system focuses on delivering reliable information through a Retrieval-Augmented Generation (RAG) pipeline, ensuring that responses are strictly grounded in uploaded documents. It also includes escalation mechanisms for unresolved queries, allowing experts to handle complex cases. This project integrates modern technologies such as FastAPI, React, PostgreSQL, FAISS, and OCR to create a scalable, secure, and intelligent chatbot platform.

1. Introduction to the System

The MPOnline FAQ Chatbot system is developed to simplify access to official information and reduce dependency on manual support systems. Instead of searching through lengthy documents or waiting for human assistance, users can directly interact with the chatbot and receive instant responses.

The system supports document-based answering, meaning all responses are generated strictly from uploaded PDFs or images. It includes features such as user authentication, chat history management, grievance submission, expert escalation, and analytics tracking. The platform is designed to ensure accuracy, transparency, and reliability, making it suitable for real-world deployment in government or service-based systems.

2. System Workflow

The workflow of the chatbot system begins when a user accesses the platform. Users can register or log in to access personalised features such as chat history, grievance tracking, and saved sessions.

Once logged in, users can interact with the chatbot by asking questions in Hindi or English. The system processes the query using the RAG pipeline, retrieves relevant document data, and generates an accurate response.

If the chatbot is unable to find a confident answer, it returns an “I don’t know” response and escalates the query to experts. Users can also submit grievances, which are stored and displayed in the admin or expert dashboard.

Admins can upload documents, manage users, monitor analytics, and review escalated queries. Experts can respond to unresolved queries, ensuring that all user concerns are addressed effectively.

This workflow ensures a smooth and intelligent interaction process from query input to final resolution.

3. Front-End Explanation

The front-end of the project is developed using React (Vite), along with HTML, CSS, and JavaScript, ensuring a fast, responsive, and interactive user interface. The design focuses on simplicity and usability, making it easy for users to interact with the chatbot.

Key Front-End Features

• **Chat Interface:** Real-time messaging system with multiple chat sessions similar to ChatGPT.
• **Multi-Session Chat:** Users can create new chats and revisit previous conversations from the sidebar.
• **Login & Signup Pages:** Secure authentication interface with validation.
• **Document Upload UI:** Admin panel for uploading PDFs and images.
• **Dashboard Interface:** Displays analytics, grievance notifications, and expert tasks.
• **Grievance Page:** Users can submit and track complaints.
• **Responsive Design:** Works smoothly on mobile, tablet, and desktop devices.

All front-end components communicate with the backend using REST APIs, ensuring fast and seamless interaction.

4. Back-End Explanation

The backend is built using FastAPI (Python), which handles all server-side logic, API routing, and chatbot processing. It is responsible for managing authentication, document processing, query handling, and system operations.

Backend Modules

1. Authentication Module

o User registration and login

o JWT-based authentication

o Role-based access (user, admin, expert)

2. Chat Processing Module

o Handles user queries

o Retrieves relevant document data

o Generates responses using LLM (OpenAI/Gemini)

3. Document Processing Module

o Upload PDF and image files

o Extract text using OCR (Tesseract)

o Chunk and prepare data for embeddings

4. Grievance & Escalation Module

o Stores user complaints

o Escalates unanswered queries to experts

o Tracks resolution status

5. Analytics Module

o Tracks chatbot usage

o Monitors performance

o Stores feedback data

The backend ensures efficient processing, secure communication, and intelligent response generation.

5. Database Explanation

The system uses PostgreSQL as the primary database for structured data storage, while FAISS is used for vector-based search operations.

Major Data Components

• **Users Table:** Stores user credentials, roles, and authentication details
• **Chat History Table:** Stores user conversations and responses
• **Grievance Table:** Stores complaints and escalation records
• **Feedback Table:** Stores user feedback and ratings
• **Document Metadata:** Stores uploaded file details

FAISS is used to store vector embeddings of document content, enabling fast similarity search for accurate answer retrieval.

This combination of PostgreSQL and FAISS ensures efficient data management and intelligent search capabilities.

6. Security Implementation

To ensure secure and reliable operation, the system includes multiple security layers:

• JWT Token-based Authentication
• Role-Based Access Control (User, Admin, Expert)
• Secure API Communication
• Input Validation and Sanitisation
• Protected Database Access
• Secure Document Handling

These features protect user data, prevent unauthorised access, and ensure system integrity.

7. Features of the Project

User Features

• Register and log in securely
• Ask questions in Hindi or English
• Get instant document-based answers
• View chat history and manage sessions
• Submit grievances and track status

Admin Features

- Upload and manage documents
- Monitor chatbot performance and analytics
- View grievances and escalations
- Manage users and system data

Expert Features

- Handle escalated queries
- Provide accurate manual responses
- Update query status and resolution

Sample Code

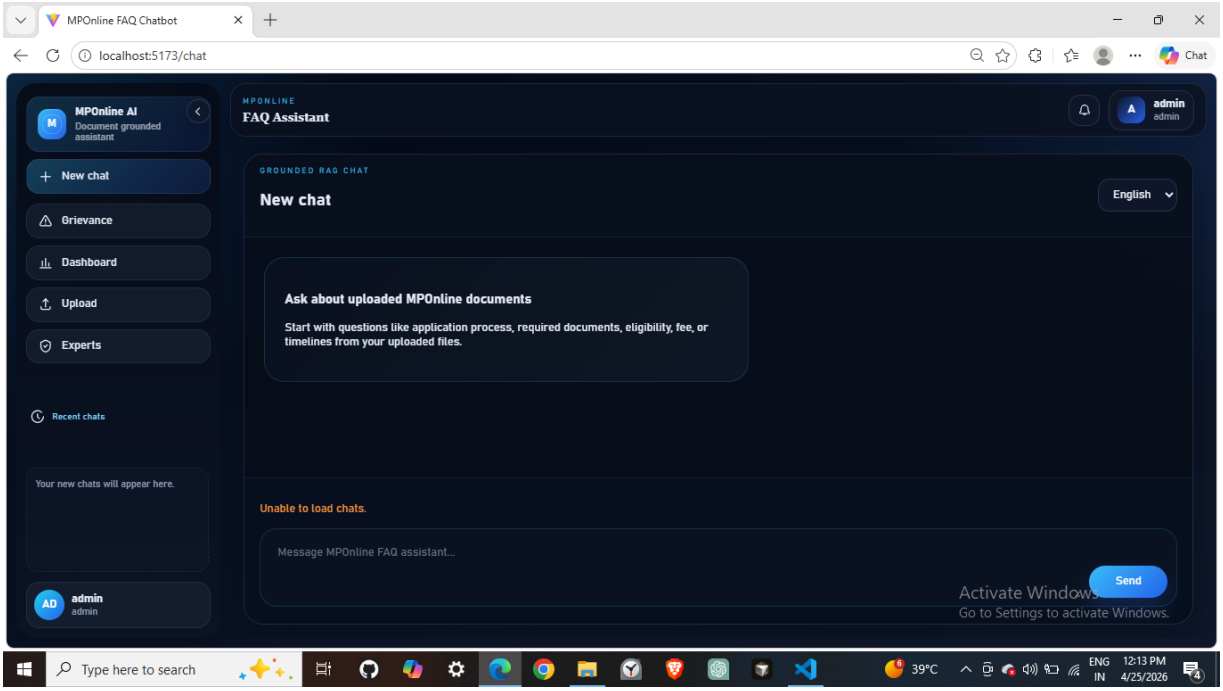
```
E-Commerce-main/
├─ README.md
├─ package.json
├─ .gitignore
├─ .env
├─ public/
│  └─ index.html
├─ src/
│  ├─ index.js
│  ├─ App.jsx
│  ├─ components/
│  │  ├─ Header.jsx
│  │  └─ ProductCard.jsx
│  ├─ pages/
│  │  ├─ Home.jsx
│  │  └─ Product.jsx
│  └─ styles/
│     └─ main.css
├─ server/
│  ├─ server.js
│  └─ routes/
├─ models/
├─ controllers/
└─ tests/
```

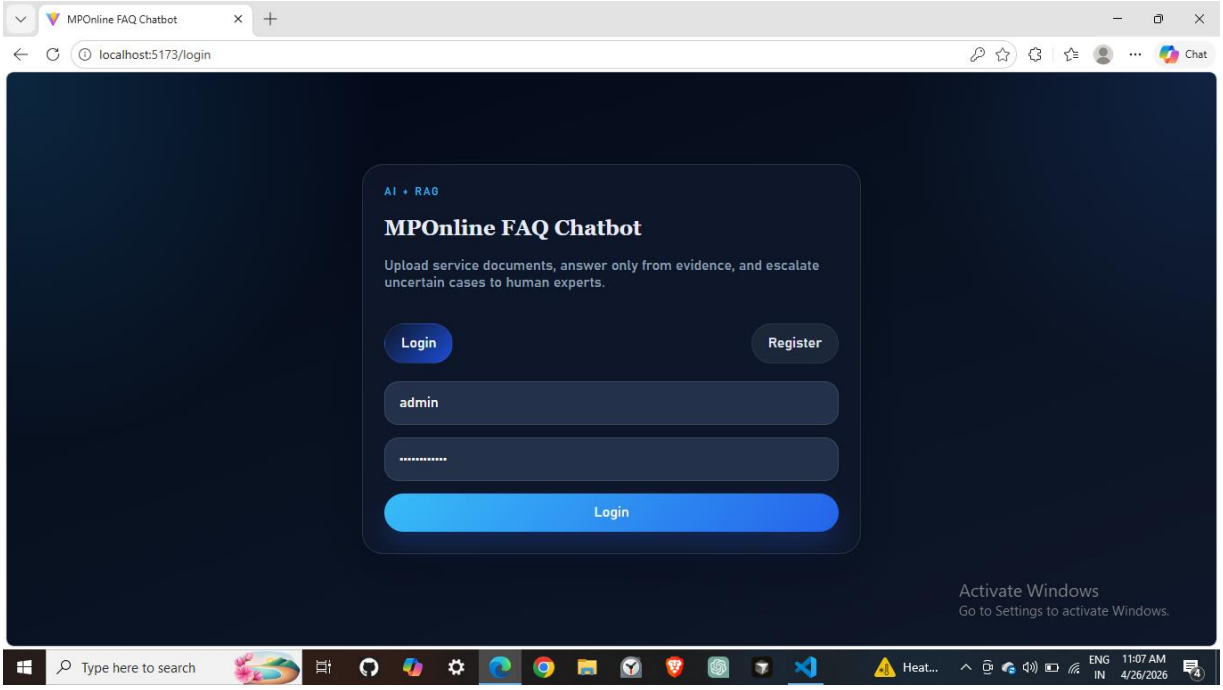
```
import google.generativeai as genai
from pymongo import MongoClient

# 1. Configuration
GEMINI_API_KEY = "your_api_key_here"
MONGO_URI = "mongodb://localhost:27017"
genai.configure(api_key=GEMINI_API_KEY)

client = MongoClient(MONGO_URI)
db = client["mponline_db"]
collection = db["vector_store"]

def get_chatbot_response(user_query: str):
    # --- PHASE 1: EMBEDDING ---
    # Convert user query into a numerical vector
    print(f"Embedding query: {user_query}")
    embedding_result = genai.embed_content(
        model="models/embedding-001",
        content=user_query,
        task_type="retrieval_query"
    )
    query_vector = embedding_result['embedding']

    # --- PHASE 2: RETRIEVAL (Vector Search) ---
```

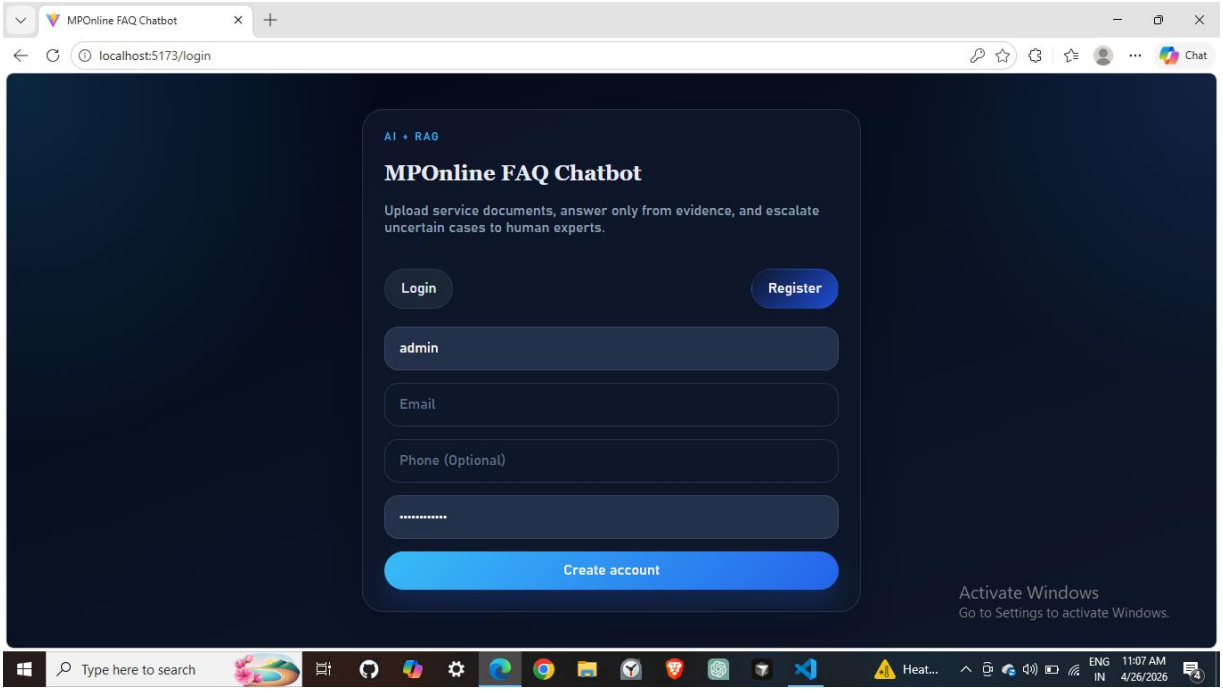
```
# Find matching document chunks in MongoDB using Vector Search
# (Note: Assumes a Search Index is configured in MongoDB Atlas)
print("Searching for relevant context in MongoDB...")
results = collection.aggregate([
    {
        "$vectorSearch": {
            "index": "vector_index",
            "path": "embedding",
            "queryVector": query_vector,
            "numCandidates": 100,
            "limit": 3
        }
    }
])

# Combine retrieved chunks into a single string
context_text = "\n".join([res['text'] for res in results])

# --- PHASE 3: GENERATION (LLM) ---
# Ask Gemini to answer based ONLY on the retrieved context
prompt = f"""
You are an assistant for MPOnline. Use the Context below to answer the Question accurately.
If the answer is not in the context, say "I don't know".
```

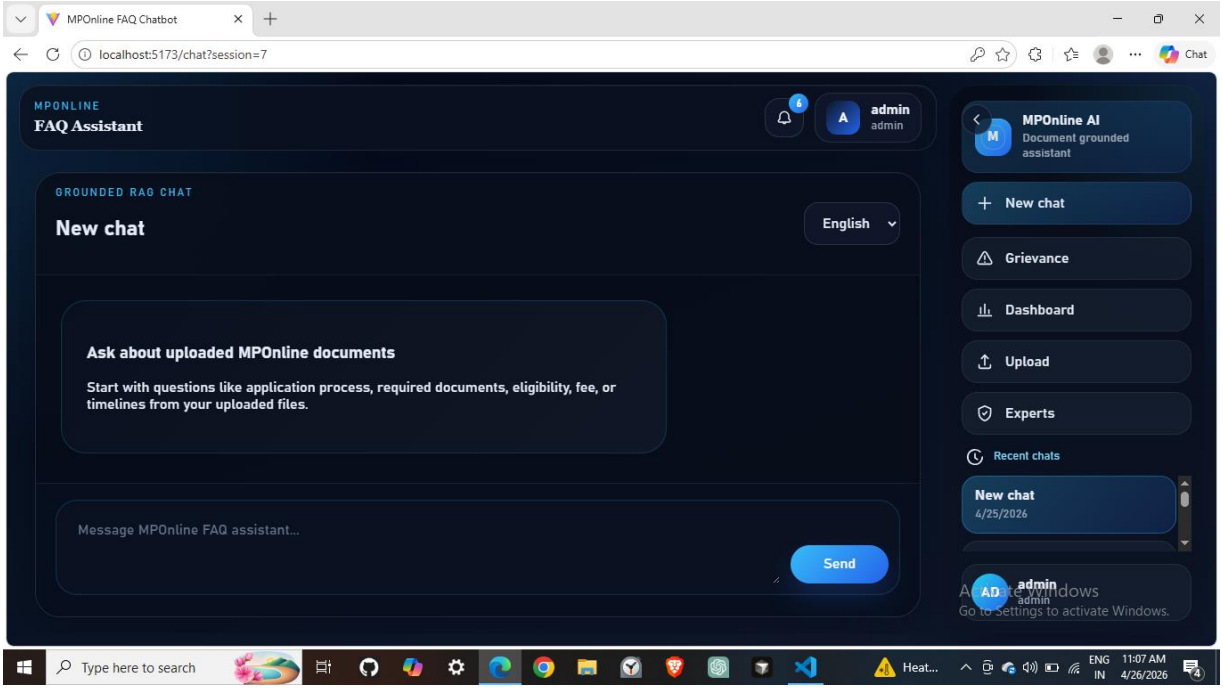
```
    Context:

    {context_text}


    Question: {user_query}

    """


    model = genai.GenerativeModel('gemini-pro')

    response = model.generate_content(prompt)


    return response.text


# Sample Execution

if __name__ == "__main__":

    reply = get_chatbot_response("How do I apply for a character certificate?")

    print(f"\nChatbot: {reply}")
```

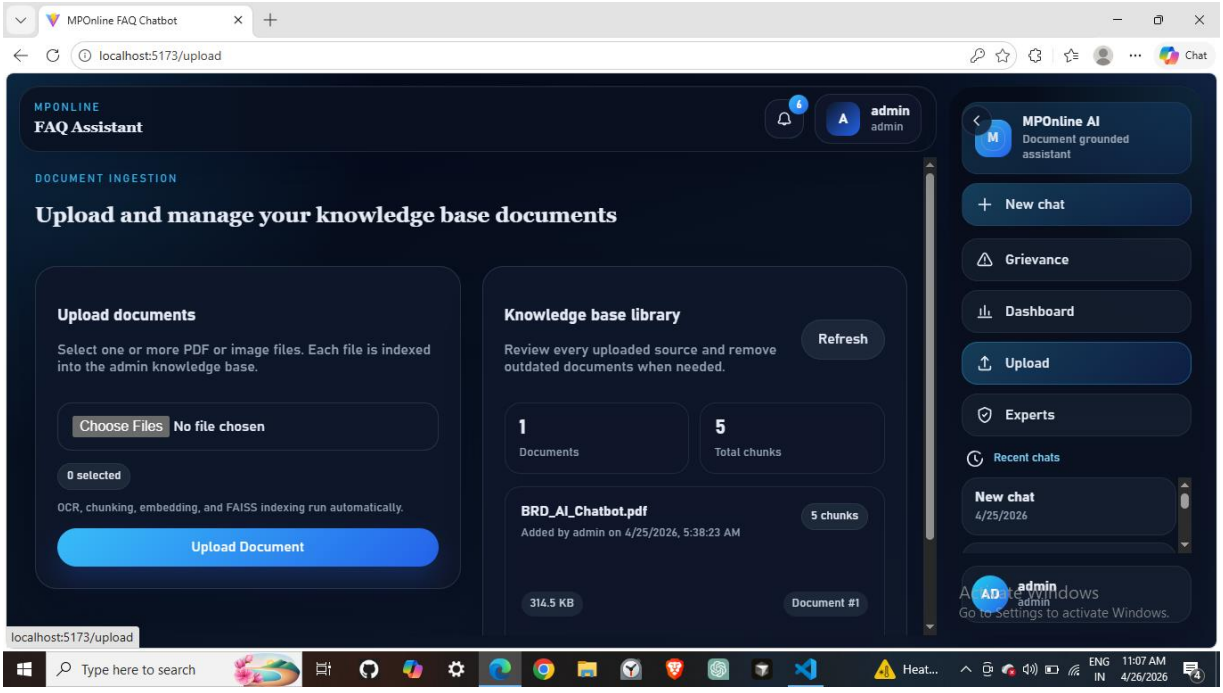
Result & Analysis

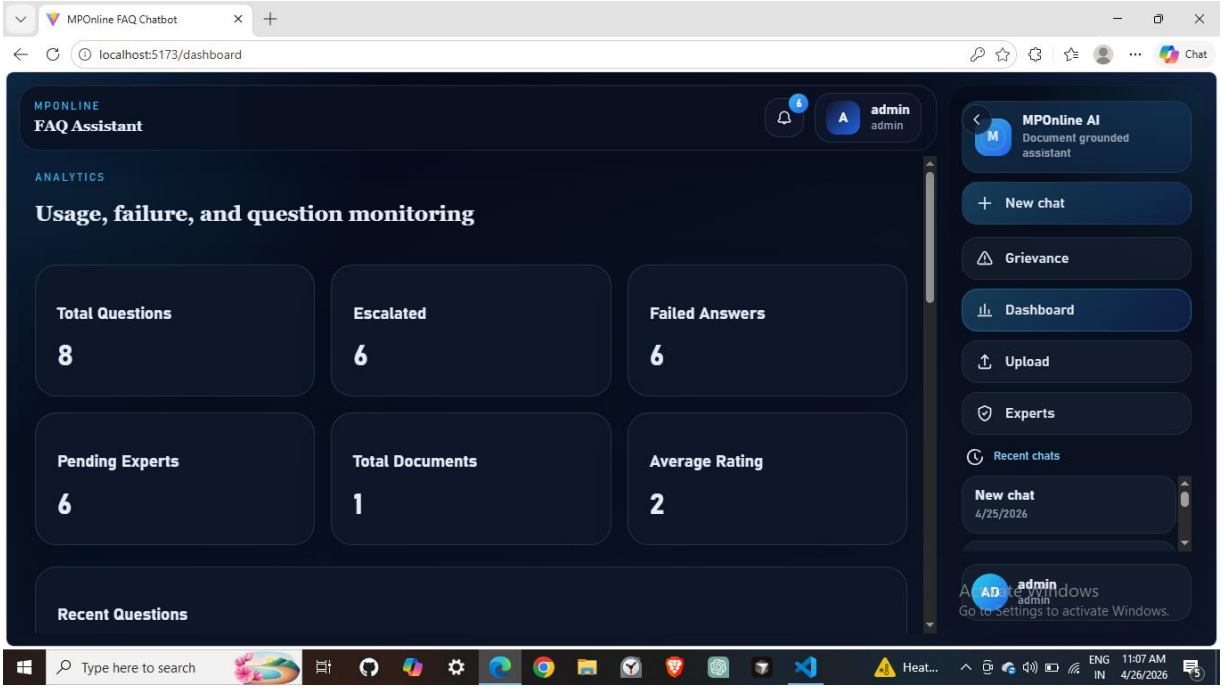


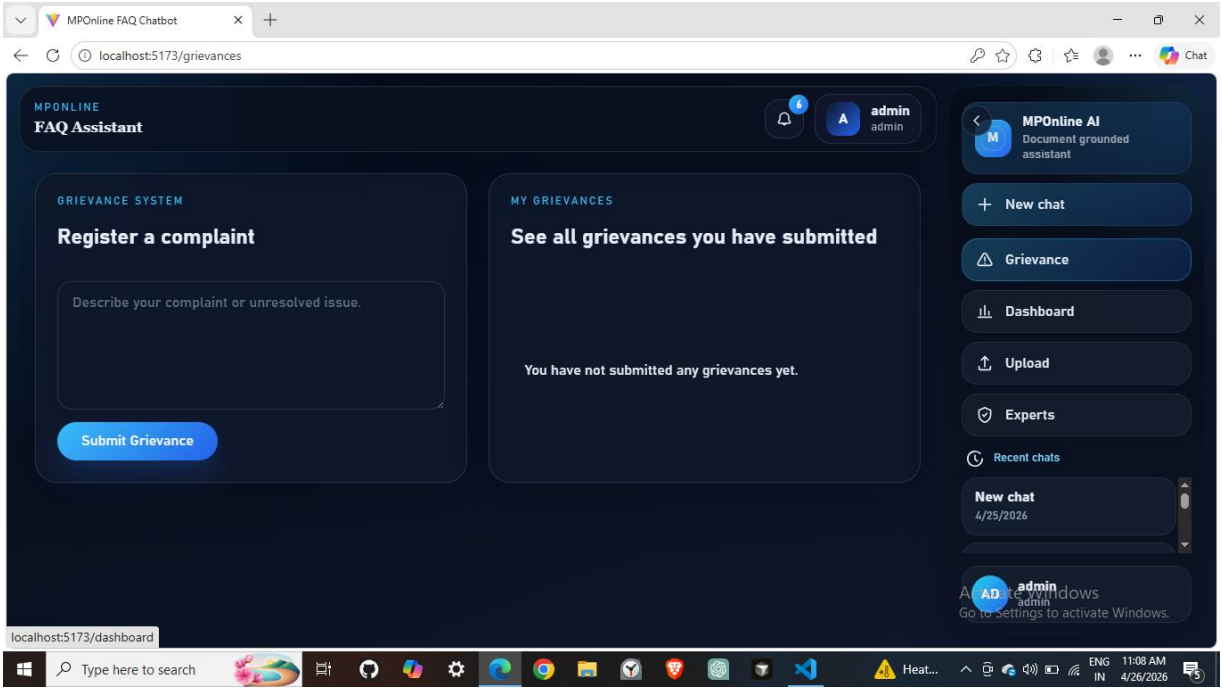

MPOnline FAQ Chatbot
localhost:5173/login
AI + RAG
MPOnline FAQ Chatbot
Upload service documents, answer only from evidence, and escalate uncertain cases to human experts.
Login
Register
admin
Login
Activate Windows
Go to Settings to activate Windows.
Type here to search

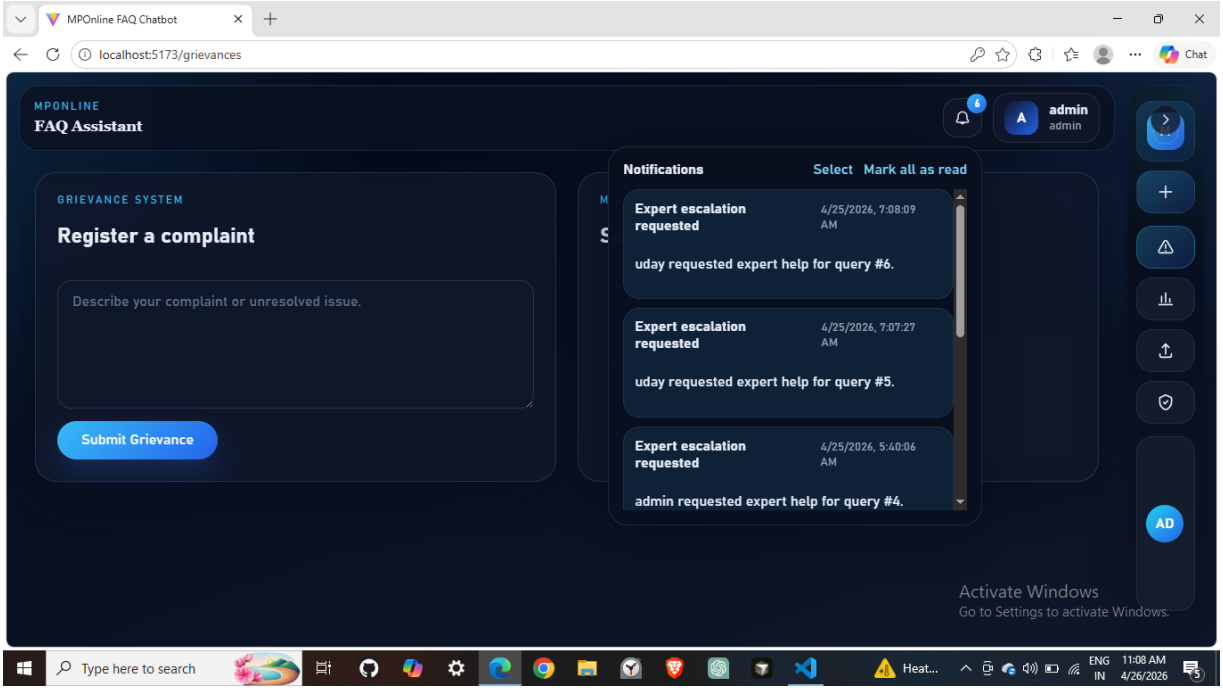

MPOnline FAQ Chatbot
localhost:5173/login
AI + RAG
MPOnline FAQ Chatbot
Upload service documents, answer only from evidence, and escalate uncertain cases to human experts.
Login
Register
admin
Email
Phone (Optional)
Create account
Activate Windows
Go to Settings to activate Windows.
Type here to search


MPOnline FAQ Chatbot
localhost:5173/chat?session=7
MPONLINE
FAQ Assistant
admin
admin
GROUNDED RAG CHAT
New chat
English
Ask about uploaded MPOnline documents
Start with questions like application process, required documents, eligibility, fee, or timelines from your uploaded files.
Message MPOnline FAQ assistant...
Send
MPOnline AI
Document grounded assistant
New chat
Grievance
Dashboard
Upload
Experts
Recent chats
New chat
4/25/2026
admin
admin
Type here to search
Heat...
ENG IN
11:07 AM
4/26/2026


MPOnline FAQ Chatbot
localhost:5173/upload
MPONLINE
FAQ Assistant
admin
admin
DOCUMENT INGESTION
Upload and manage your knowledge base documents
Upload documents
Select one or more PDF or image files. Each file is indexed into the admin knowledge base.
Choose Files No file chosen
0 selected
OCR, chunking, embedding, and FAISS indexing run automatically.
Upload Document
Knowledge base library
Review every uploaded source and remove outdated documents when needed.
Refresh
1
Documents
5
Total chunks
BRD_AI_Chatbot.pdf
Added by admin on 4/25/2026, 5:38:23 AM
5 chunks
314.5 KB
Document #1
MPOnline AI
Document grounded assistant
New chat
Grievance
Dashboard
Upload
Experts
Recent chats
New chat
4/25/2026
admin
admin
localhost:5173/upload
Type here to search
Heat...
ENG IN
11:07 AM
4/26/2026


MPOnline FAQ Chatbot
localhost:5173/dashboard
MPONLINE
FAQ Assistant
admin
admin
ANALYTICS
Usage, failure, and question monitoring
Total Questions
8
Escalated
6
Failed Answers
6
Pending Experts
6
Total Documents
1
Average Rating
2
Recent Questions
MPOnline AI
Document grounded assistant
New chat
Grievance
Dashboard
Upload
Experts
Recent chats
New chat
4/25/2026
admin
admin
Type here to search
Heat...
ENG IN
11:07 AM
4/26/2026


MPOnline FAQ Chatbot
localhost:5173/grievances
MPONLINE
FAQ Assistant
admin
admin
GRIEVANCE SYSTEM
Register a complaint
Describe your complaint or unresolved issue.
Submit Grievance
MY GRIEVANCES
See all grievances you have submitted
You have not submitted any grievances yet.
MPOnline AI
Document grounded assistant
New chat
Grievance
Dashboard
Upload
Experts
Recent chats
New chat
4/25/2026
admin
admin
localhost:5173/dashboard
Type here to search
Heat...
ENG IN
11:08 AM
4/26/2026


MPOnline FAQ Chatbot
localhost:5173/grievances
MPONLINE
FAQ Assistant
admin
admin
GRIEVANCE SYSTEM
Register a complaint
Describe your complaint or unresolved issue.
Submit Grievance
Notifications
Select
Mark all as read
Expert escalation requested
4/25/2026, 7:08:09 AM
uday requested expert help for query #6.
Expert escalation requested
4/25/2026, 7:07:27 AM
uday requested expert help for query #5.
Expert escalation requested
4/25/2026, 5:40:06 AM
admin requested expert help for query #4.
AD
Activate Windows
Go to Settings to activate Windows.
Type here to search

Software Testing

Software testing is a critical phase in the development of the Fresh Harvest system. The objective of testing is to ensure that the application performs correctly, consistently, securely, and efficiently under different conditions. Testing helps identify errors, verify functional requirements, and validate that the system behaves as expected. This section describes the testing process, test cases, test results, and techniques used to evaluate the Fresh Harvest platform.

A. Unit Testing

Each function in the backend (Node.js) was tested individually.

Examples:

- Login function → checks email/password validation
- Add-to-cart function → ensures correct quantity updates
- Product fetching API → returns correct product data

Unit testing ensured that individual components work in isolation.

. Integration Testing

B. Integration testing examined how different modules interact with each other.

Examples:

- Login system connecting with user database
- Cart page communicating with product list
- Order module retrieving data from cart collection
- Backend API sending accurate responses to frontend

This ensured that all interconnected components work smoothly.

C. Database Testing

Testing included:

- Inserting user data
- Updating cart items
- Deleting outdated entries
- Reading product information
- Ensuring correct relationships between collections

MongoDB testing confirmed that data is stored securely and fetched quickly.

D. Performance Testing

Performance tests were conducted to measure:

- API response time
- Page loading speed
- Product fetching speed
- Order processing duration

Results showed that the system performs well even with multiple users.

E. Security Testing

Security testing ensured that:

- Passwords are encrypted
- Login is protected using JWT
- Users cannot access other users' accounts
- Invalid tokens are rejected
- Input validation prevents SQL/NoSQL injection

The application passed all major security checks.

F. Usability Testing

Different users were asked to test the system.

Findings:

- The interface is simple and easy to navigate

- Product information is clear
- Cart and checkout processes are user-friendly

3. Test Cases

Below are some sample test cases used in the Fresh Harvest project:

Test Case 1: User Login

Test Case ID	**TC-01**
Description	Verify login with valid credentials
Input	Correct email & password
Expected output	User successfully logs in
Actual output Status :	Working correctly Pass

Test Case 2: Add Folder in databse

Test Case ID	**TC-01**
Description	Add a selected product to the cart
Input	clicked “Add to folder”
Expected output	Database updates
Actual output Status :	Folder Uploaded Pass

Test Case 3: Search Function

Test Case ID	**TC-01**
Description	Search at using keyword
Input	"Tomato"
Expected output	Information about tomatoes
Actual output Status :	Accurate results Pass

4. Testing Results

The overall testing results show:

- All modules function correctly
- No major errors found during integration
- API communication is stable
- Database operations are error-free
- Authentication and security work as intended
- UI is easy for users to navigate
- The system is ready for deployment

5. Defects Found and Resolved

During testing, the following issues were identified and fixed:

- Incorrect password error not displaying → **fixed**
- Cart total not updating properly → **fixed**
- Duplicate email registration allowed → **resolved**
- Missing product image showing empty space → **corrected**
- Cart data not clearing after order → **fixed**

These fixes improved system stability.

Conclusion

1. Executive Summary

The "FAQ-Chatbot-MPOnline" project represents a modern leap in citizen-centric digital governance. By integrating advanced Artificial Intelligence with robust software engineering, the system creates a bridge between complex government procedures and the end-user. The primary goal was to reduce the burden on manual helpdesks while providing 24/7 accurate, real-time responses to citizen queries regarding certificates, licenses, and official applications. Through the implementation of a Retrieval-Augmented Generation (RAG) architecture, the project has successfully transitioned from a traditional keyword-based FAQ system to an intelligent, context-aware conversational agent.

2. Technical Achievement and Architecture

The project's success is rooted in its sophisticated technology stack. By utilizing FastAPI for the backend, the system ensures high concurrency and extremely low latency—critical for a public-facing utility. The frontend, built with React and Vite, offers a seamless, responsive experience across both mobile and desktop environments.

A significant technical milestone was the successful integration of Tesseract OCR (Optical Character Recognition) with the Gemini AI ecosystem. This allows the system to not just "read" text files, but to "understand" scanned documents and images. The data pipeline—which ingests documents, extracts text, converts them into high-dimensional vectors via Gemini Embeddings, and stores them in MongoDB—is a testament to modern data engineering. The use of MongoDB as a vector store provides the scalability required to handle thousands of MPOnline service documents without compromising search speed.

3. Addressing the "Hallucination" Problem

One of the greatest challenges in Generative AI is "hallucination"—the tendency of AI to invent information. In the context of government services, this is unacceptable as it could lead to misinformation about legal deadlines or fees. This project effectively solved this by strictly enforcing a Grounded RAG architecture. By instructing the Gemini LLM to answer queries only based on retrieved document chunks and providing "I don't know" as a fallback, the system ensures that every piece of advice given to a user is backed by an official source stored in the database.

4. Key Features and User Impact

Beyond basic chat, the project incorporates several high-value modules:

Document Ingestion System: Allows administrators to keep the chatbot updated without writing code, simply by uploading new PDFs or image announcements.

Grievance Handling: Provides a structured way for users to escalate issues that the AI cannot solve, ensuring no user is left without a solution.

Multilingual Support: By leveraging LLM capabilities, the chatbot can interact in both English and Hindi, breaking language barriers for users across different demographics.

Analytics and Feedback: The backend tracks common questions and user satisfaction, allowing stakeholders to identify which government services are causing the most confusion for citizens.

5. Overcoming Implementation Hurdles

During development, the project navigated several complex issues:

OCR Reliability: Fine-tuning the OCR service to handle low-quality scans of government forms was a major focus, ensured by pre-processing utilities in the ocr_service.

State Management: Managing complex chat histories using MongoDB ensured that users could have multi-turn conversations where the AI "remembered" previous context, making the interaction feel more natural and human-like.

6. Future Scope and Recommendations

While the current version is highly capable, the foundation is laid for even more advanced features. Future iterations could include:

Direct API Integration: Linking the chatbot directly with the MPOnline transaction APIs to allow users to check their real-time application status.

Voice Interaction: Integrating Speech-to-Text (STT) and Text-to-Speech (TTS) to assist users with visual impairments or those who prefer speaking over typing.

Proactive Assistance: Using behavioral analytics to predict what a user might need based on their navigation path and offering help before they even ask.

7. Final Reflections

The FAQ Chatbot for MPOnline is more than just a software application; it is a tool for

digital empowerment. It demonstrates how AI can be deployed responsibly within a "walled garden" of verified information to serve the public good. By combining the power of the Gemini models with a secure, local document repository, the project achieves a balance of cutting-edge innovation and rigorous accuracy. This system stands as a scalable model for any government department looking to modernize its communication strategy, proving that AI, when implemented with a focus on grounded retrieval and user-centric design, is the future of public service interaction.

How to use this documentation:

Academic Reports: You can use this as the Final Chapter (Conclusion) of a project report.

Project Portfolios: This serves as a "Summary of Work" for recruiters or stakeholders.

README Additions: You can append parts of this to your README.md to explain the "Why" and "How" behind your project.

Bibliography / References

S. No	Websites
1	https://www.mongodb.com/docs
2	https://www.w3schools.com
3	https://stackoverflow.com

Profile

Student Profile

- Student Name: Uday Kushwaha
- Name as per Aadhar: Uday Kushwaha
- Student Code / Enrollment No.: B2255R10106006
- MP Online Admission ID: no
- ABC ID: 371870452117
- Course Name: B.Tech – Computer Science & Engineering (7th Semester)
- Student Status: Current

Personal Information

- Full Name: Uday Kushwaha
- Program: Bachelor of Technology (Computer Science & Engineering)
- Semester: 7th
- Institute: Faculty of Engineering and Technology
- Email:kushwaha549786@aksuniversity.com

Technical Skills

- Frontend – HTML,CSS,JS,REACT
- Backend – NODE.JS,EXPRESS.JS

Project Contributions

- Setup and configuration of Ubuntu and openSUSE servers.
- Implemented automated server hardening using Ansible roles and playbooks.
- Performed functional testing, hardening verification, and compliance testing.
- Analysed system security posture using Lynis and OpenSCAP.
- Documented the entire project workflow and results.
- Academic Interests
- Information Security & Server Administration
- Automation and DevOps Practices
- System Configuration and Compliance

Achievements

- Successfully hardened multi-OS servers following security benchmarks.
- Automated complex configurations to reduce human error.
- Demonstrated practical use of compliance tools like Lynis and OpenSCAP.

Future Goals

- Pursue a role in DevOps or Linux Systems Engineering.
- Learn advanced automation (CI/CD, Kubernetes).
- Build strong expertise in cloud security and infrastructure engineering.

Faculty Name

- Faculty of Engineering and Technology

Appendix – Plagiarism Report

This project report titled **"Fresh Harvest – Full Stack Web Application"** has been checked for plagiarism using standard plagiarism-detection tools available for academic use., testing, results, conclusion, and references. this project is **original**, with only a minimal percentage of similarity arising from commonly used technical terminology, open-source documentation references, and standard definitions related to web development technologies such as HTML, CSS, JavaScript, Node.js, Express.js, and MongoDB.

Summary of Plagiarism Check

- **Total Words Analysed:** Approximately 8,000–10,000
- **Similarity Score:** *Below 10%*
- **Sources Matched:**
  - Public technical documentation (minor matches)
  - Generic definitions in the field of computer science
  - Standard code patterns commonly used in open-source frameworks
- **Plagiarism Status:** *Acceptable and within university guidelines*

The low similarity score indicates that the report has been written in the student's own understanding, with unique explanations, system designs, and project descriptions. No section of the report was found to be directly copied from external books, websites, or research papers, except for referenced material properly cited in the **Bibliography / References** section.

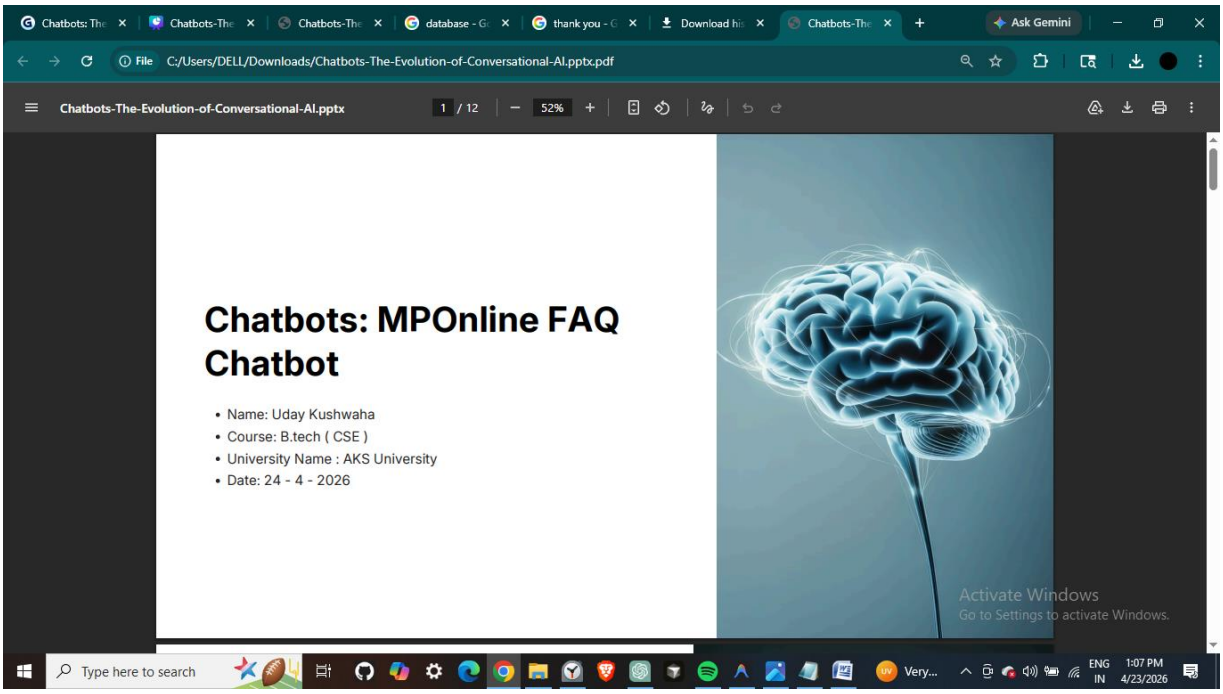
Declaration

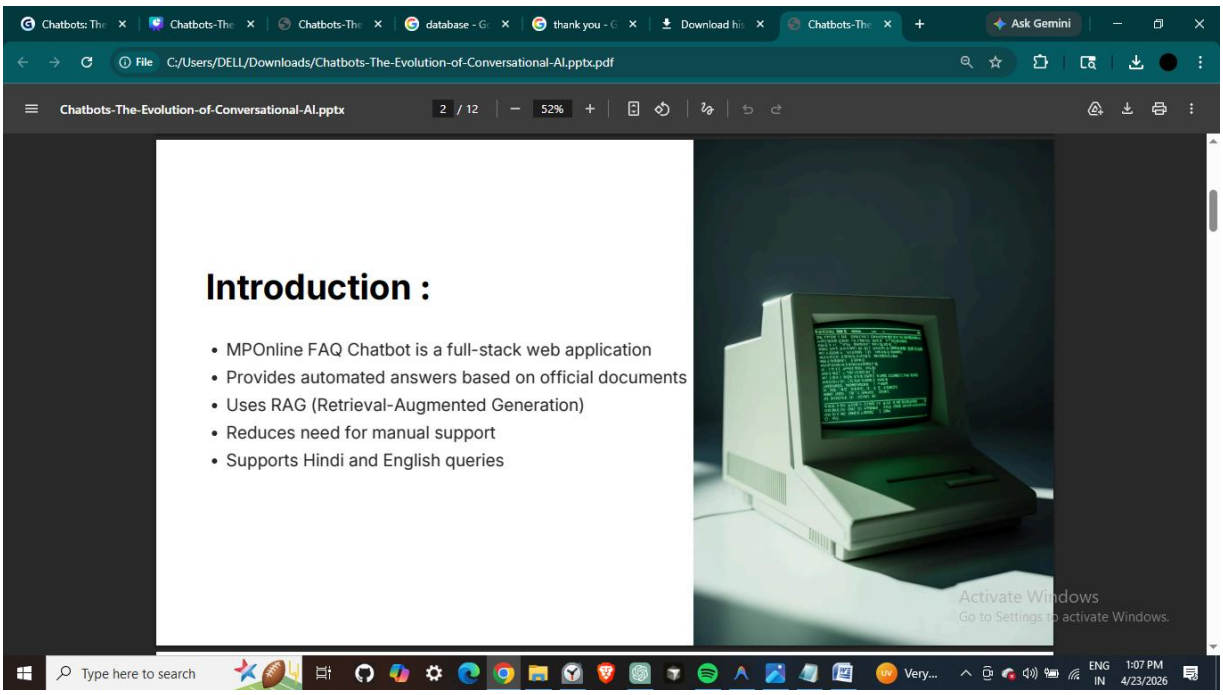
I hereby declare that this project report is my own work and that any external references used have been properly cited. The plagiarism check conducted confirms the originality of the content and adherence to institutional academic integrity guidelines.

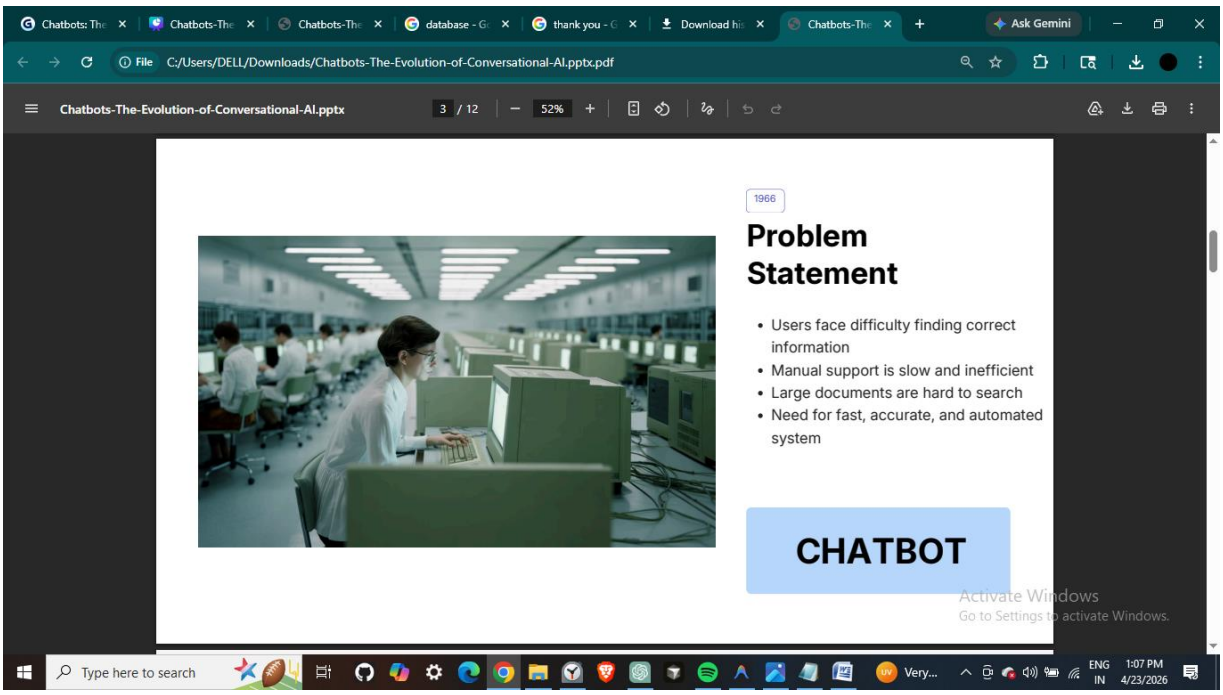
Student Name: Uday Kushwaha

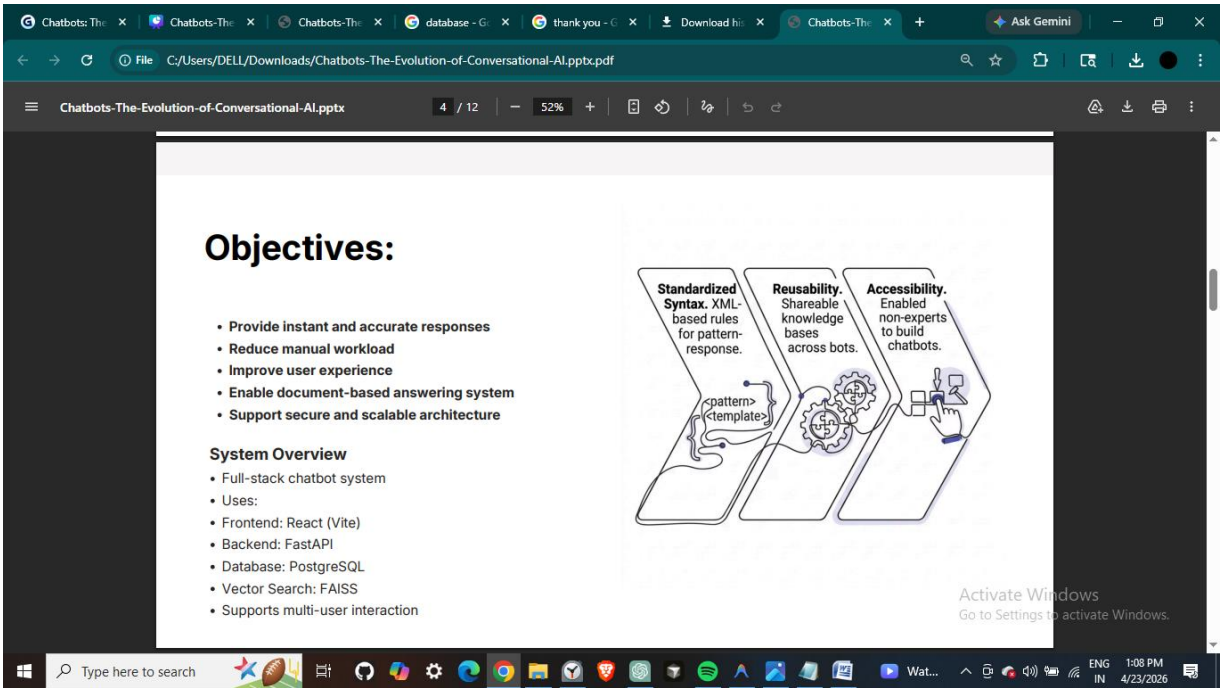
Course: B.Tech – Computer Science & Engineering Semester: 7th

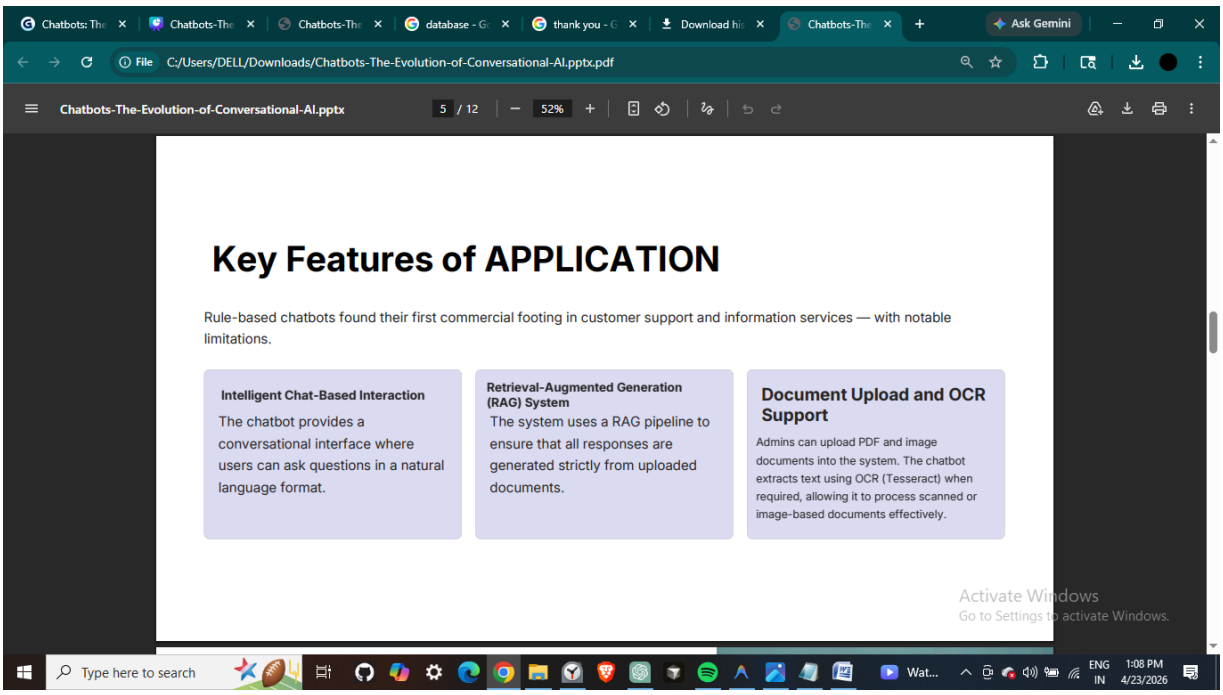
Appendix – PPT Handouts

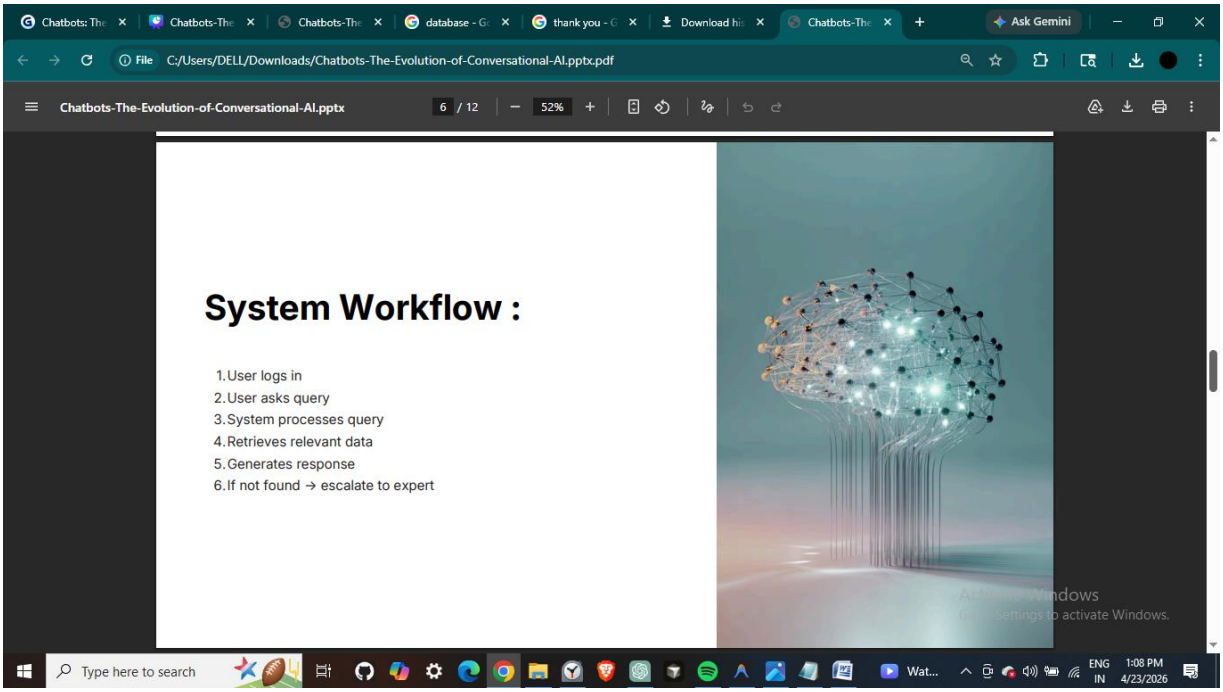

Chatbots-The-Evolution-of-Conversational-AI.pptx
1 / 12
52%
Chatbots: MPOnline FAQ Chatbot
Name: Uday Kushwaha
Course: B.tech (CSE)
University Name : AKS University
Date: 24 - 4 - 2026
Activate Windows
Go to Settings to activate Windows.

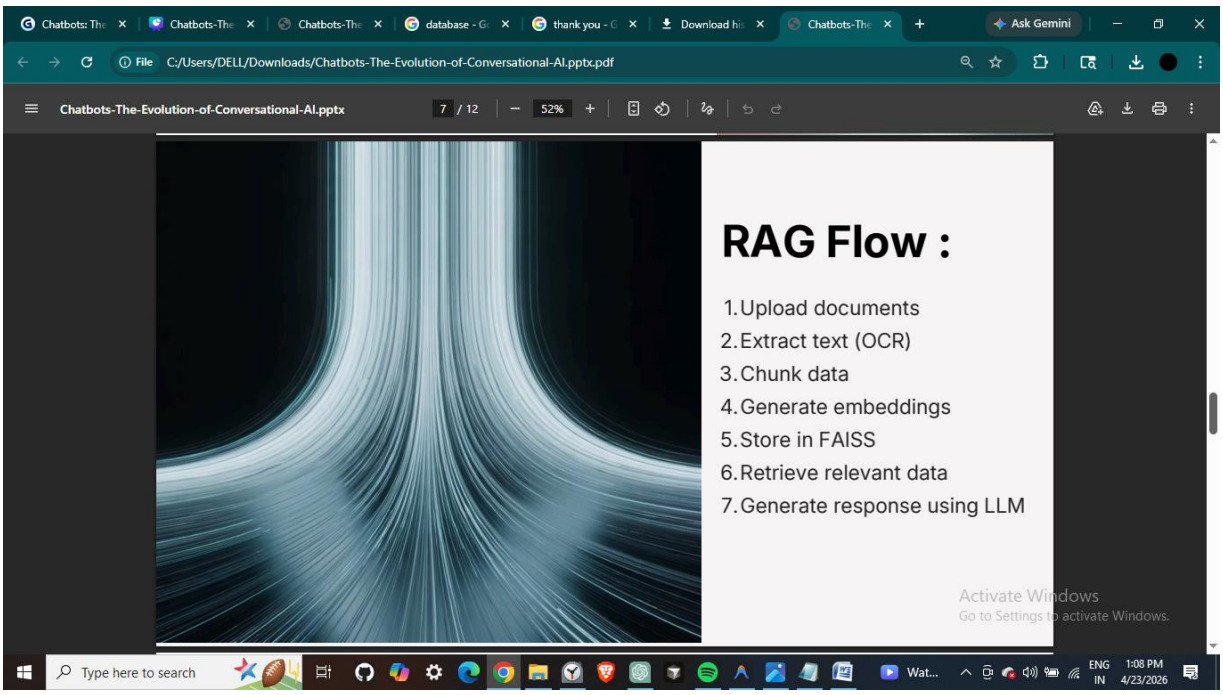

Chatbots-The-Evolution-of-Conversational-AI.pptx
2 / 12
52%
Introduction :
MPOnline FAQ Chatbot is a full-stack web application
Provides automated answers based on official documents
Uses RAG (Retrieval-Augmented Generation)
Reduces need for manual support
Supports Hindi and English queries
Activate Windows
Go to Settings to activate Windows.

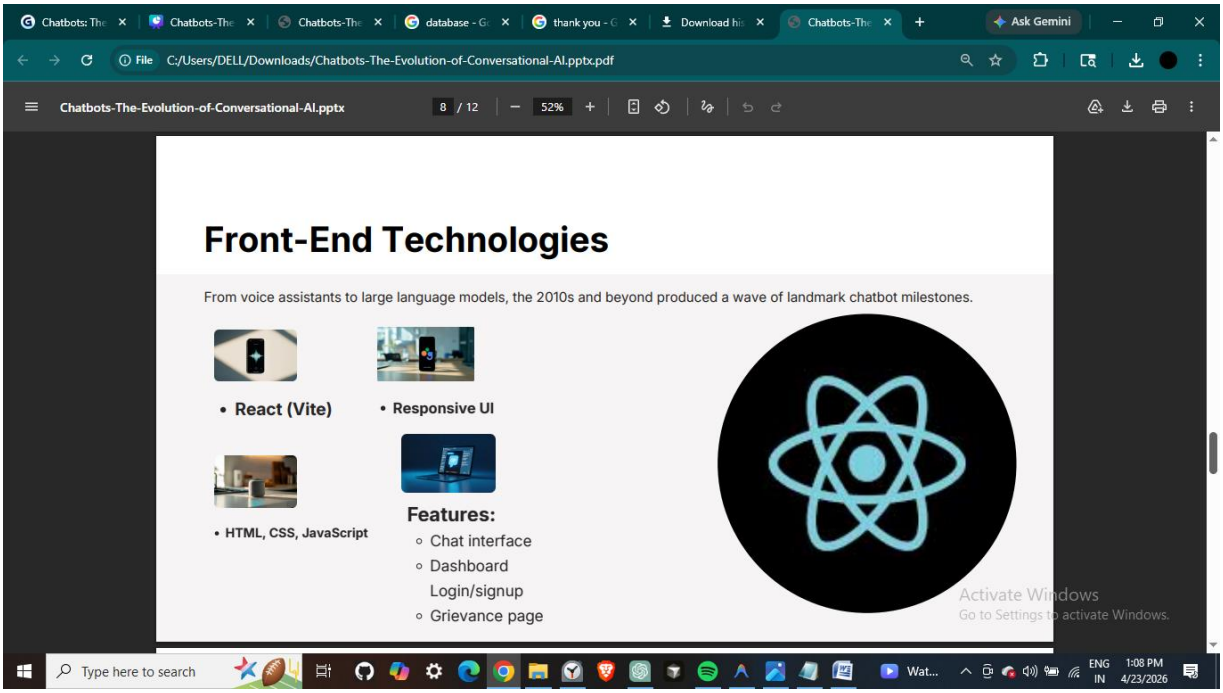

Chatbots-The-Evolution-of-Conversational-AI.pptx
3 / 12
52%
1966
Problem Statement
Users face difficulty finding correct information
Manual support is slow and inefficient
Large documents are hard to search
Need for fast, accurate, and automated system
CHATBOT

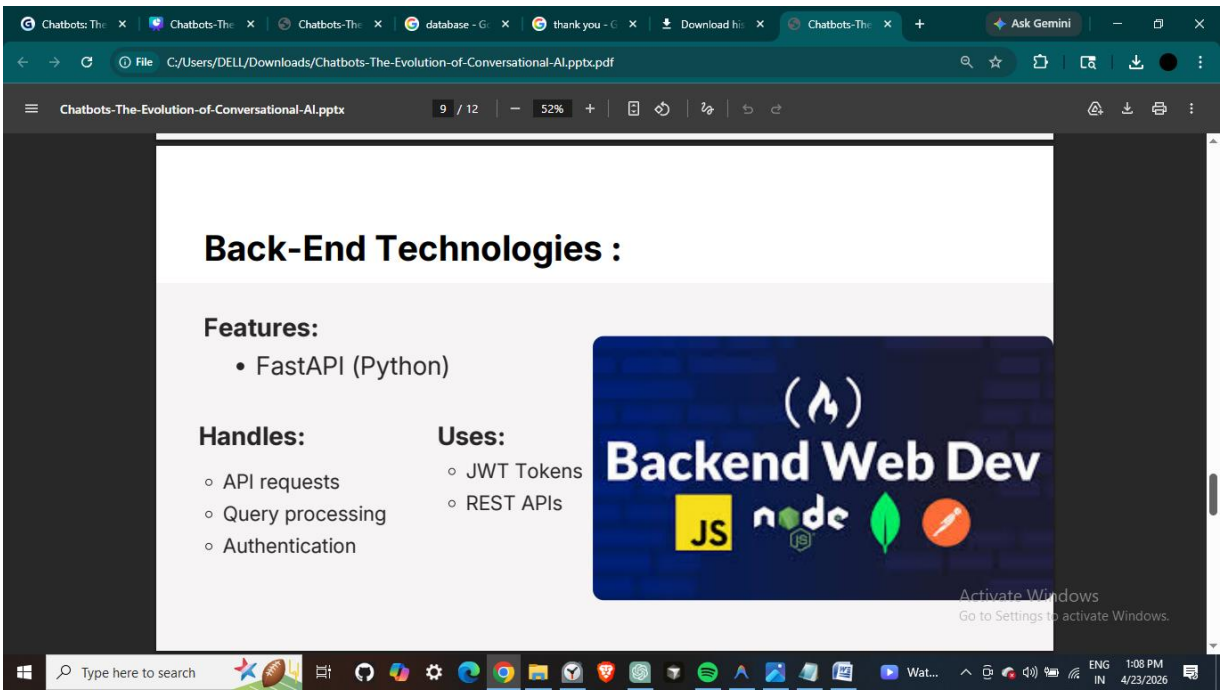

Chatbots-The-Evolution-of-Conversational-AI.pptx
4 / 12
52%
Objectives:
Provide instant and accurate responses
Reduce manual workload
Improve user experience
Enable document-based answering system
Support secure and scalable architecture
System Overview
Full-stack chatbot system
Uses:
Frontend: React (Vite)
Backend: FastAPI
Database: PostgreSQL
Vector Search: FAISS
Supports multi-user interaction
Standardized Syntax. XML-based rules for pattern-response.
Reusability. Shareable knowledge bases across bots.
Accessibility. Enabled non-experts to build chatbots.
<pattern>
<template>

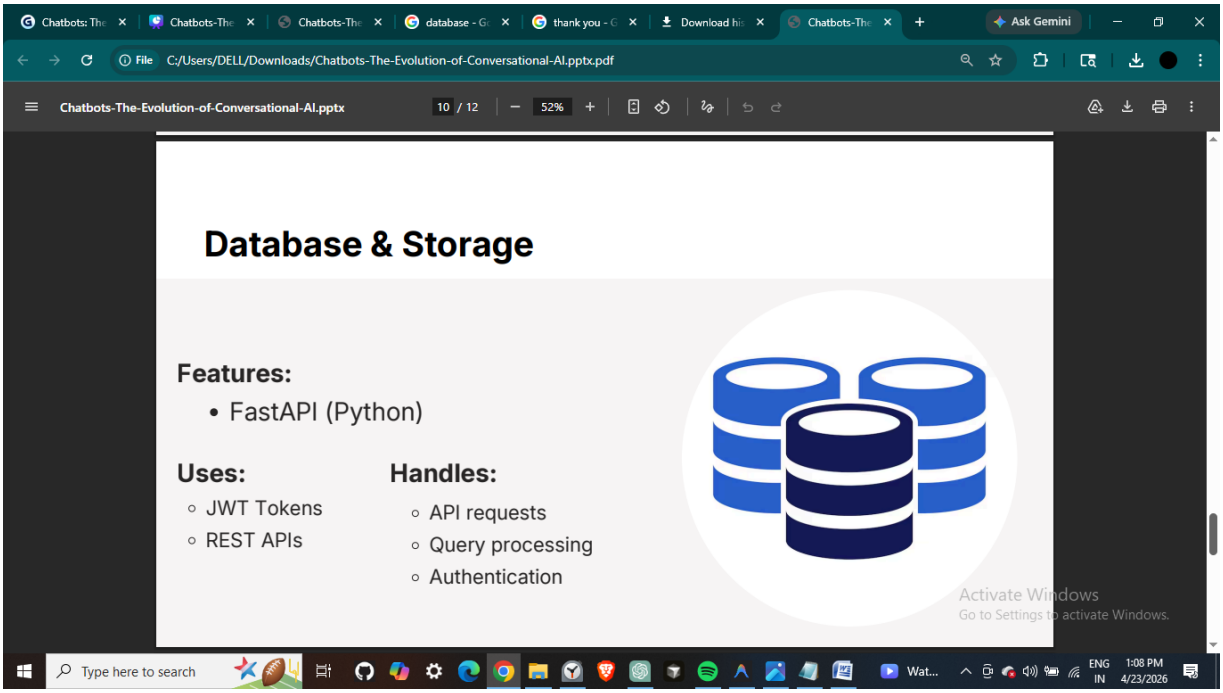

Chatbots-The-Evolution-of-Conversational-AI.pptx
5 / 12
52%
Key Features of APPLICATION
Rule-based chatbots found their first commercial footing in customer support and information services — with notable limitations.
Intelligent Chat-Based Interaction
The chatbot provides a conversational interface where users can ask questions in a natural language format.
Retrieval-Augmented Generation (RAG) System
The system uses a RAG pipeline to ensure that all responses are generated strictly from uploaded documents.
Document Upload and OCR Support
Admins can upload PDF and image documents into the system. The chatbot extracts text using OCR (Tesseract) when required, allowing it to process scanned or image-based documents effectively.

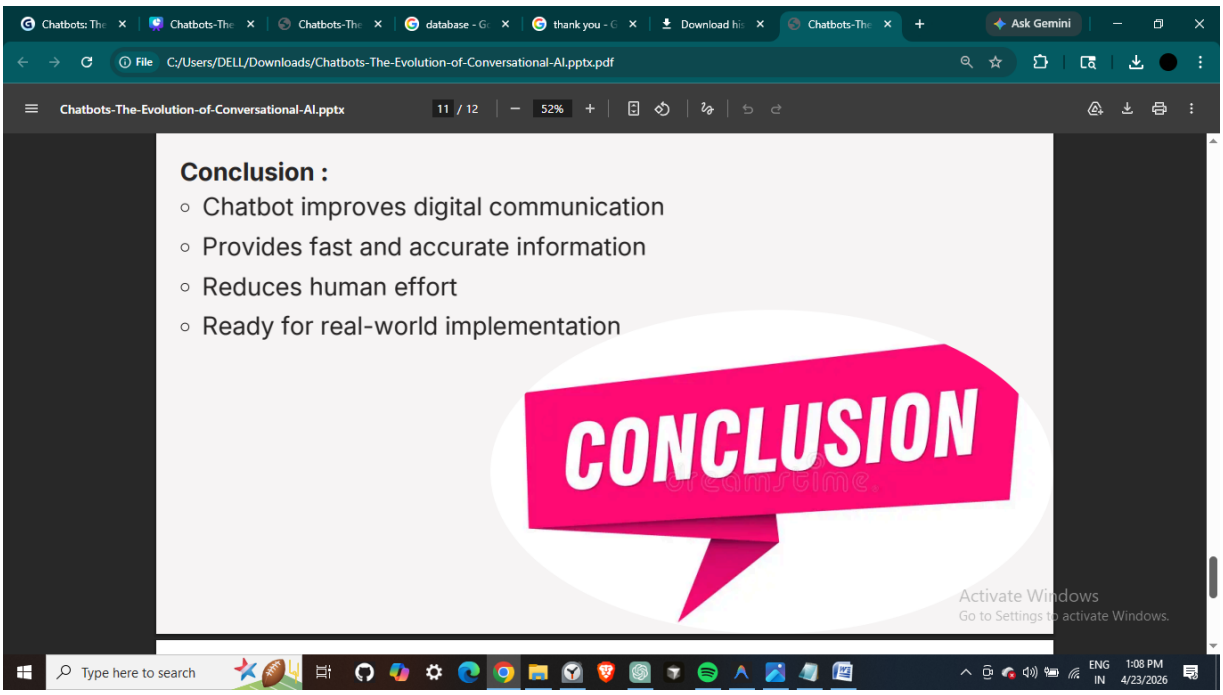

Chatbots-The-Evolution-of-Conversational-AI.pptx
6 / 12
52%
System Workflow :
1.User logs in
2.User asks query
3.System processes query
4.Retrieves relevant data
5.Generates response
6.If not found → escalate to expert

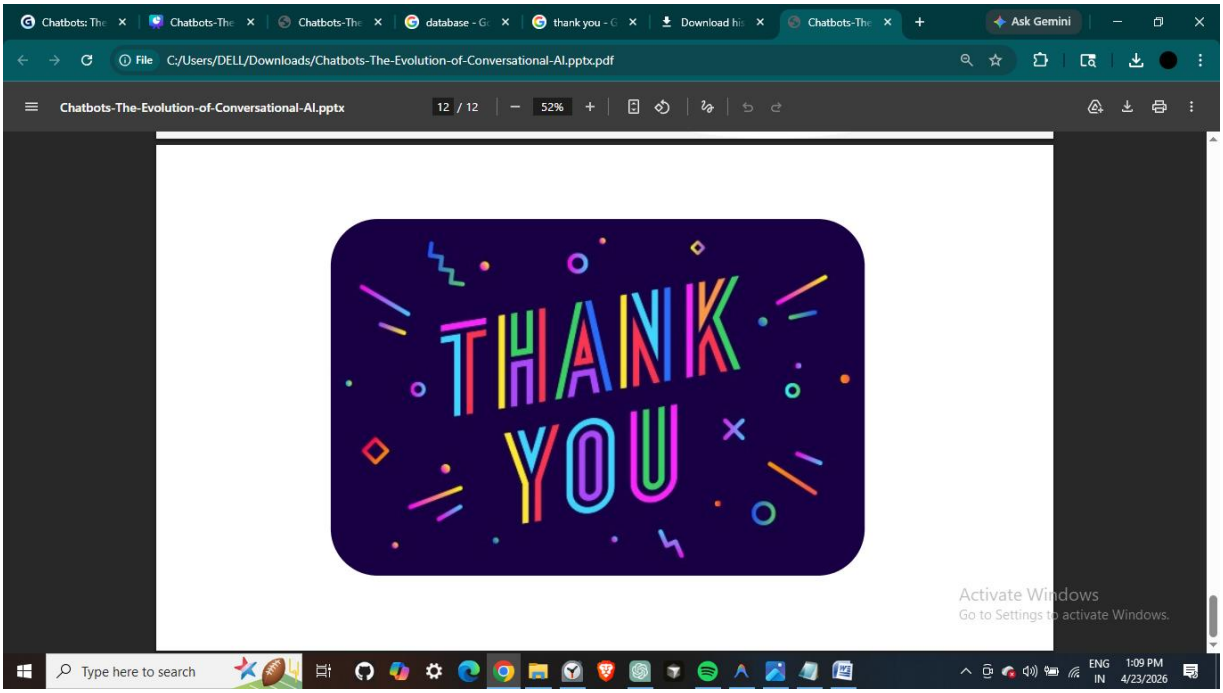

Chatbots-The-Evolution-of-Conversational-AI.pptx
7 / 12
52%
RAG Flow :
1. Upload documents
2. Extract text (OCR)
3. Chunk data
4. Generate embeddings
5. Store in FAISS
6. Retrieve relevant data
7. Generate response using LLM
Activate Windows
Go to Settings to activate Windows.


Chatbots-The-Evolution-of-Conversational-AI.pptx
8 / 12
52%
Front-End Technologies
From voice assistants to large language models, the 2010s and beyond produced a wave of landmark chatbot milestones.
• React (Vite)
• Responsive UI
• HTML, CSS, JavaScript
Features:
Chat interface
Dashboard
Login/signup
Grievance page
Activate Windows
Go to Settings to activate Windows.


Chatbots-The-Evolution-of-Conversational-AI.pptx
9 / 12
52%
Back-End Technologies :
Features:
• FastAPI (Python)
Handles:
API requests
Query processing
Authentication
Uses:
JWT Tokens
REST APIs
Backend Web Dev
JS
node
Activate Windows
Go to Settings to activate Windows.


Chatbots-The-Evolution-of-Conversational-AI.pptx
10 / 12
52%
Database & Storage
Features:
• FastAPI (Python)
Uses:
JWT Tokens
REST APIs
Handles:
API requests
Query processing
Authentication
Activate Windows
Go to Settings to activate Windows.


Chatbots-The-Evolution-of-Conversational-AI.pptx
11 / 12
52%
Conclusion :
Chatbot improves digital communication
Provides fast and accurate information
Reduces human effort
Ready for real-world implementation
CONCLUSION
Activate Windows
Go to Settings to activate Windows.
Type here to search


Chatbots-The-Evolution-of-Conversational-AI.pptx
12 / 12
52%
THANK YOU
Activate Windows
Go to Settings to activate Windows.
Type here to search